

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по теме:
**РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И
МОНИТОРИНГА СЕРВИСОВ-ЗАГЛУШЕК В СРЕДЕ
НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Медведев Клим Николаевич, 221-329 /
подпись *ФИО, группа*

Руководитель ВКР: _____ / Кружалов Алексей Сергеевич /
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная инженерия»

Тема ВКР	
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	
Основные функции	1. Список функций
Используемые технологии и платформы	Перечисление технологий.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Список задач
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Кружалов Алексей Сергеевич /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Медведев Клим Николаевич, 221-329 /
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная работа (далее ВКР) на тему: «Тема».

Цель ВКР – ...ОИИАОИ

Наполнение аннотации смотрите в методичке.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	12
1.1 Исследование инфраструктуры нагрузочного тестирования и роли имитационных сервисов	12
1.2 Анализ существующих подходов к управлению сервисами-заглушками и проблема импортозамещения	13
1.3 Сравнительный анализ инструментальных средств разработки и обоснование технологического стека	15
1.4 Обзор аналогичных решений и обоснование разработки собственного программного комплекса	21
1.5 Формирование функциональных и технических требований к системе	28
1.6 Выводы по первой главе	32
2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ УПРАВЛЕНИЯ	34
2.1 Разработка архитектуры распределённого программного комплекса	34
2.2 Проектирование структур данных для хранения конфигураций и состояний в Redis	42
2.3 Разработка макетов приложения	49
2.4 Реализация асинхронных механизмов управления процессами и ввода пользовательских параметров.	54
2.5 Разработка модуля динамического проксирования запросов на базе FastAPI и алгоритма Rate Limiting	58
2.6 Проектирование и программная реализация пользовательского интерфейса	63
2.7 Реализация механизма экспорта метрик мониторинга	71
2.8 Выводы по второй главе	77

3	ВНЕДРЕНИЕ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО КОМПЛЕКСА	80
3.1	Описание процесса развертывания системы в среде ОС Astra Linux (настройка Uvicorn/Gunicorn и службы Redis).	80
3.2	Тестирование механизмов автоматического восстановления состояний (персистентность Redis AOF) и перезапуска сервисов.	80
3.3	Экспериментальная оценка производительности прокси-модуля и эффективности алгоритма Rate Limiting под нагрузкой.	80
3.4	Анализ работы системы в кластерном режиме и оценка алгоритмов распределения нагрузки между узлами.	80
3.5	Рекомендации по дальнейшему развитию системы (шаблонизация конфигураций, Self-healing).	80
3.6	Выводы по третьей главе.	80
	ЗАКЛЮЧЕНИЕ	81
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	82
	ПРИЛОЖЕНИЕ А	85

ВВЕДЕНИЕ

Актуальность темы. В современной практике обеспечения качества сложных программных комплексов проведение нагрузочного тестирования требует создания стабильной и управляемой среды. Одной из ключевых проблем при организации такой среды является управление инфраструктурой имитационных сервисов (заглушек) – специализированного программного обеспечения, воспроизводящего поведение реальных внешних компонентов системы.

В условиях реализации политики импортозамещения в Российской Федерации актуальность приобретает разработка отечественных инструментов управления инфраструктурой, способных заменить зарубежные программные решения и обеспечить совместимость с сертифицированными отечественными операционными системами.

Цель работы – проектирование и разработка автоматизированной информационной системы для централизованного управления и мониторинга состояния распределённой инфраструктуры имитационных сервисов.

Задачи исследования:

- а) Провести анализ предметной области и существующих подходов к управлению имитационными средами в процессах обеспечения качества ПО.
- б) Спроектировать архитектуру программного комплекса, обеспечивающую гибкое масштабирование и работу в различных режимах.
- в) Обосновать выбор технологического стека и программных средств реализации системы.
- г) Разработать алгоритмы динамического проксирования трафика, механизмы контроля интенсивности запросов и программные интерфейсы для автоматизации управления.
- д) Реализовать систему хранения конфигураций и контроля состояний исполняемых файлов.

е) Провести тестирование разработанного комплекса в среде ОС Astra Linux и оценить эффективность реализованных механизмов мониторинга и управления.

Объект исследования – инфраструктура запуска и эксплуатации имитационных сервисов (заглушек) в среде нагрузочного тестирования.

Предмет исследования – механизмы и алгоритмы автоматизированного управления и контроля состояния имитационных сервисов в среде нагрузочного тестирования.

Научная новизна работы заключается в разработке алгоритма централизованного управления распределённой средой, поддерживающего индивидуальную настройку ограничений интенсивности запросов (Rate Limiting) для каждого имитационного сервиса в отдельности, а также в реализации агентно-независимой схемы взаимодействия с удалёнными узлами посредством протоколов SSH/SCP без установки дополнительного программного обеспечения на целевые серверы.

Практическая значимость работы состоит в создании программного инструмента, позволяющего отказаться от использования зарубежных систем управления, упростить эксплуатацию инфраструктуры имитационных сервисов и обеспечить её интеграцию со стандартными средствами мониторинга (Prometheus).

Положения, выносимые на защиту:

а) Архитектура программного комплекса, поддерживающая гибкое масштабирование системы от локального запуска на одном хосте до формирования распределённого кластера.

б) Механизм динамического проксирования запросов, реализующий прозрачную маршрутизацию входящего трафика к целевым имитационным сервисам.

в) Алгоритм контроля интенсивности запросов (Rate Limiting), обеспечивающий стабильность работы имитационной среды при пиковых нагрузках.

г) Методика формирования и экспорта метрик производительности для непрерывного мониторинга состояния инфраструктуры заглушек.

Структура работы. Выпускная квалификационная работа включает введение, три главы основного текста, заключение, список использованных источников и приложения.

Во введении обоснована актуальность исследования, поставлены цель и задачи, определены объект и предмет работы, указаны научная новизна и практическая значимость полученных результатов.

В первой главе проводится анализ предметной области, исследуются существующие аналоги и методы управления имитационными средами. На основе сравнительного анализа обосновывается выбор технологий и формируются требования к системе.

Во второй главе описывается проектирование архитектуры системы, структуры хранения данных и логики работы основных модулей. Приводятся схемы взаимодействия компонентов и алгоритмы реализации функций проксирования и управления кластером.

В третьей главе рассматриваются вопросы практической реализации, настройки и эксплуатации системы в среде ОС Astra Linux. Приводятся результаты тестирования, подтверждающие корректность работы разработанных механизмов управления и мониторинга.

В заключении сформулированы основные выводы, подтверждающие решение поставленных задач и достижение цели работы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исследование инфраструктуры нагрузочного тестирования и роли имитационных сервисов

В современной практике обеспечения качества сложных программных комплексов нагрузочное тестирование занимает критически важное место. Оно позволяет гарантировать стабильность и управляемость информационной системы при высоких эксплуатационных нагрузках. Одной из ключевых проблем при организации таких испытаний является необходимость взаимодействия объекта тестирования с множеством внешних систем и сервисов.

Использование реальных внешних интеграций в среде тестирования часто оказывается невозможным или нецелесообразным по ряду причин:

- ограниченность ресурсов: внешние системы могут иметь жёсткие лимиты на количество запросов или требовать значительных финансовых затрат на эксплуатацию;
- изоляция среды: для получения достоверных результатов необходимо исключить влияние задержек и сбоев сторонних систем на показатели исследуемого объекта;
- риски изменения данных: проведение тестов на реальных интеграциях может привести к нежелательному изменению или повреждению информации в продуктовых базах данных.

Для решения этих проблем применяются имитационные сервисы («заглушки») – специализированное программное обеспечение, которое воспроизводит поведение реальных компонентов, отвечая на запросы тестируемой системы заранее определённым образом. В контексте данной работы в качестве имитационных сервисов рассматриваются Java-приложения в форматах `.jar` и `.war`.

Роль имитационных сервисов в инфраструктуре нагрузочного тестирования заключается в следующем:

- обеспечение автономности стенда: заглушки позволяют полностью изолировать тестируемую систему от внешнего окружения, создавая предсказуемую среду;
- эмуляция различных сценариев: имитационные сервисы могут воспроизводить не только корректные ответы, но и ошибки (HTTP 5xx, тайм-ауты), что необходимо для проверки механизмов отказоустойчивости;
- контроль интенсивности трафика: использование механизмов ограничения запросов (Rate Limiting) позволяет моделировать поведение внешних систем при достижении ими предельных порогов производительности.

Однако при увеличении масштаба информационной системы количество необходимых заглушек возрастает, что порождает проблему управления распределённой инфраструктурой. Процессы развёртывания, контроля состояния (активна/остановлена) и маршрутизации трафика к десяткам имитационных сервисов на различных серверах в ручном режиме существенно замедляют подготовку к испытаниям.

Таким образом, для эффективного функционирования стендов нагрузочного тестирования необходима специализированная система. Она должна централизованно управлять жизненным циклом заглушек, обеспечивать их мониторинг и динамическое проксирование запросов, что особенно актуально в условиях перехода на отечественные операционные системы, такие как ОС Astra Linux.

1.2 Анализ существующих подходов к управлению сервисами-заглушками и проблема импортозамещения

Традиционным подходом к управлению имитационными сервисами (заглушками), разработанными на языке Java, является их развёртывание в специализированных контейнерах серверов приложений. В современной

практике разработки и тестирования сложился ряд стандартных решений, среди которых наиболее распространёнными являются:

- Apache Tomcat – наиболее лёгкий и популярный сервлет-контейнер, используемый для запуска веб-приложений. Он обеспечивает базовую поддержку спецификаций Java Servlet и JavaServer Pages;

- WildFly (ранее JBoss Application Server) – более мощная и полнофункциональная платформа, поддерживающая широкий спектр стандартов Java EE (Jakarta EE). Данное решение предоставляет расширенные возможности управления ресурсами и кластеризации[1];

- Eclipse GlassFish – эталонная реализация платформы Java EE, предлагающая развитые инструменты администрирования через графическую консоль и командную строку[2].

Анализ данных решений позволяет выделить общие тенденции и характеристики классических серверов приложений:

- универсальность и полнофункциональность: все перечисленные платформы спроектированы для запуска сложных корпоративных информационных систем. Однако для узкоспециализированной задачи – работы лёгковесных имитационных заглушек – их функциональность является избыточной;

- значительные накладные расходы: общим недостатком является высокое потребление оперативной памяти и ресурсов центрального процессора самой платформой. При необходимости запуска десятков или сотен заглушек на одном или нескольких хостах суммарные затраты ресурсов на поддержание среды выполнения становятся критическими;

- сложность автоматизации и оркестрации: управление большим парком разрозненных экземпляров серверов приложений требует внедрения дополнительных тяжеловесных систем автоматизации. Штатные средства управления (консоли администрирования) ориентированы на ручное конфигурирование, что затрудняет их использование в динамических сценариях нагрузочного тестирования;

— монолитная архитектура управления: классические подходы подразумевают жёсткую привязку приложения к конкретной конфигурации сервера, что снижает гибкость при необходимости быстрого изменения параметров запуска (например, JVM-аргументов) для отдельных сервисов.

На текущий момент в российской ИТ-индустрии наблюдается отчётливая тенденция к отказу от использования зарубежных проприетарных и открытых решений в пользу отечественного программного обеспечения или собственных разработок. Это обусловлено политикой импортозамещения и необходимостью обеспечения технологического суверенитета.

Большинство западных систем управления инфраструктурой и серверов приложений (таких как продукты Oracle или Red Hat) сталкиваются с проблемами поддержки, обновления и совместимости с российскими операционными системами. В частности, эксплуатация имитационных стендов в защищённой среде ОС Astra Linux требует инструментов, которые нативно поддерживают механизмы управления этой операционной системой и не зависят от зарубежных репозиторий и лицензий.

Таким образом, анализ существующих подходов показывает, что использование традиционных серверов приложений типа Tomcat или WildFly для управления распределённой сетью заглушек становится неэффективным. Современные требования диктуют необходимость перехода к более лёгким, гибким и отечественным решениям, способным обеспечить централизованное управление жизненным циклом сервисов в рамках распределённых кластеров.

1.3 Сравнительный анализ инструментальных средств разработки и обоснование технологического стека

Выбор программных средств для реализации системы управления имитационными сервисами является фундаментальным этапом проектирования. От корректности этого выбора зависят эксплуатационные характеристики бу-

дущей системы: предельная пропускная способность, латентность (время отклика), масштабируемость и устойчивость к пиковым нагрузкам.

В ходе анализа предметной области и целей исследования было определено, что разрабатываемая система должна обеспечивать бесперебойную оркестрацию интенсивного потока запросов от нагрузочных станций. Это накладывает жёсткие ограничения на архитектуру: управляющий контур не должен становиться «узким местом» (bottleneck) при маршрутизации трафика. Требование минимизации накладных расходов исключает использование технологий, склонных к блокировке вычислительных ресурсов при ожидании ответа от внешних систем.

Для обоснования выбора проводится многокритериальный анализ по трём направлениям: язык программирования, веб-фреймворк и система управления базами данных.

1.3.1 Обоснование выбора языка программирования серверной части

Для разработки backend-составляющей системы рассматривались три языка программирования, наиболее востребованных в сфере высоконагруженных систем (HighLoad) и системной автоматизации.

Java — объектно-ориентированный язык программирования, являющийся отраслевым стандартом для корпоративных систем и «родной» средой для запуска самих имитационных сервисов (.jar).

Преимущества: строгая статическая типизация, снижающая количество ошибок на этапе компиляции; развитая экосистема библиотек и инструментов профилирования; поддержка полноценной многопоточности (multithreading).

Недостатки: высокая ресурсоёмкость виртуальной машины (JVM), требующая значительного объёма оперативной памяти; длительное время «холодного старта», что критично для динамических сред тестирования; гро-

моздкий синтаксис для простых задач взаимодействия с процессами операционной системы.

Go (Golang) – компилируемый многопоточный язык программирования, разработанный Google, ориентированный на создание эффективных сетевых сервисов.

Преимущества: высочайшая производительность, сопоставимая с C++, благодаря компиляции в нативный код; эффективная модель конкурентности (goroutines), позволяющая обрабатывать тысячи соединений с минимальными накладными расходами; отсутствие внешних зависимостей (компиляция в единый бинарный файл).

Недостатки: отсутствие гибкости при работе с динамическими структурами данных (JSON произвольной структуры) из-за строгой типизации; специфическая обработка ошибок и отсутствие исключений, что увеличивает объём шаблонного кода (boilerplate); более высокий порог входа по сравнению с интерпретируемыми языками.

Python – высокоуровневый интерпретируемый язык программирования общего назначения с динамической строгой типизацией.

Преимущества: наличие мощных инструментов асинхронного программирования (библиотека `asyncio`[3]), позволяющих эффективно обрабатывать задачи, ограниченные скоростью ввода-вывода (I/O-bound), такие как сетевые запросы или ожидание ответа от базы данных; глубокая интеграция с Linux: модуль `subprocess` предоставляет удобный интерфейс для управления процессами, сигналами и потоками ввода-вывода; высокая скорость разработки благодаря лаконичному синтаксису.

Недостатки: производительность интерпретатора ниже, чем у компилируемых языков; наличие Global Interpreter Lock (GIL), ограничивающего выполнение потоков на одном ядре CPU (нивелируется использованием асинхронного подхода).

Вывод: в качестве основного языка выбран Python. Возможность использования асинхронного программирования в сочетании с нативными

средствами управления процессами Linux позволяет обеспечить необходимую производительность, сохраняя гибкость и скорость разработки.

1.3.2 Сравнительный анализ веб-фреймворков

Учитывая необходимость обеспечения высокой пропускной способности системы, выбор архитектуры веб-фреймворка является критическим. Рассматривались три кандидата: Django, Flask и FastAPI.

Django – полнофункциональный синхронный фреймворк уровня Enterprise, следующий принципу «Batteries included» (всё включено).

Преимущества: наличие встроенной панели администрирования, ORM (Object-Relational Mapping) и системы аутентификации «из коробки»; развитая экосистема плагинов и высокий уровень безопасности; строгая структура проекта, облегчающая поддержку крупных монолитных приложений.

Недостатки: монолитная архитектура и избыточный функционал создают значительные накладные расходы (overhead) на каждый запрос; синхронная модель обработки запросов плохо подходит для высоконагруженных микросервисов, так как блокирует потоки при ожидании ввода-вывода; сложность отключения неиспользуемых компонентов для облегчения системы.

Flask – лёгковесный микрофреймворк, классическое решение, основанное на спецификации WSGI (Web Server Gateway Interface).

Преимущества: минимализм и модульность: разработчик подключает только необходимые библиотеки; низкий порог входа и простота создания простых сервисов; гибкость в выборе архитектурных паттернов.

Недостатки: использование синхронной (блокирующей) модели ввода-вывода – при высокой нагрузке это приводит к исчерпанию пула потоков сервера и росту задержек; отсутствие встроенных механизмов валидации данных (требует подключения сторонних расширений); снижение производи-

тельности при большом количестве одновременных соединений (проблема C10k).

FastAPI[4] – современный асинхронный фреймворк, построенный на спецификации ASGI (Asynchronous Server Gateway Interface)[5].

Преимущества: полная поддержка асинхронности (`async/await`) и событийного цикла, что позволяет обрабатывать тысячи запросов в секунду на одном процессе; высокая производительность благодаря использованию Starlette и uvloop, сопоставимая с Go и Node.js; встроенная быстрая валидация данных и сериализация на базе Pydantic[6].

Недостатки: относительно молодая экосистема по сравнению с Django и Flask; требует понимания принципов асинхронного программирования.

Вывод: выбран фреймворк FastAPI [4], [7]. Это единственное решение в экосистеме Python, способное обеспечить стабильную обработку интенсивного трафика с минимальными задержками благодаря асинхронной неблокирующей архитектуре.

1.3.3 Обоснование выбора системы управления данными (СУБД)

Для обеспечения высокой отзывчивости системы подсистема хранения данных должна обладать минимальным временем отклика на чтение конфигураций и запись статусов. Сравнивались конкретные распространённые решения.

PostgreSQL – объектно-реляционная СУБД с открытым исходным кодом.

Преимущества: гарантия целостности данных (ACID) и надёжность хранения; мощный язык запросов SQL и поддержка сложных выборок (JOIN); широкие возможности расширяемости (типы данных, индексы).

Недостатки: накладные расходы на дисковые операции ввода-вывода и журналирование (WAL) замедляют отклик; избыточность функционала для

хранения простых пар «ключ-значение» (конфигурации заглушек); потребность в строгом проектировании схемы данных (Schema migrations).

MySQL – реляционная СУБД, популярная база данных, часто используемая в веб-приложениях.

Преимущества: высокая скорость операций чтения в простых сценариях; широкая распространённость и простота администрирования; поддержка различных движков хранения (Storage Engines).

Недостатки: как и любая реляционная СУБД, зависит от скорости дисковой подсистемы; блокировки таблиц или строк могут снижать конкурентность записи при высокой нагрузке; жёсткость схемы данных.

MongoDB – документоориентированная NoSQL СУБД, система, хранящая данные в формате BSON (бинарный JSON).

Преимущества: гибкая схема данных (Schemaless), идеально подходящая для хранения JSON-конфигураций; горизонтальная масштабируемость (шардинг) из коробки.

Недостатки: потребляет значительно больше памяти и дискового пространства по сравнению с аналогами; в стандартной конфигурации уступает In-Memory решениям по скорости отклика; слабые гарантии транзакционности в старых версиях.

Redis – In-Memory хранилище структур данных[8], высокопроизводительная СУБД, работающая полностью в оперативной памяти.

Преимущества: экстремально низкая латентность (время доступа менее 1 мс) благодаря отсутствию обращений к диску при чтении; простая модель данных «Ключ-Значение», полностью соответствующая задаче хранения состояний процессов; нативная поддержка механизма Time-To-Live (TTL) для временных данных.

Недостатки: зависимость объёма хранимых данных от объёма оперативной памяти; риск потери данных при аварийном отключении питания (решается настройкой персистентности).

Вывод: выбрана СУБД Redis[8]. Использование In-Memory архитектуры обеспечивает максимальное быстродействие. Проблема сохранности данных решается включением режима AOF (Append Only File)[9], который асинхронно сохраняет операции на диск, гарантируя восстановление конфигураций после перезагрузки без ущерба для производительности.

1.3.4 Итоговый выбор средств разработки

На основании проведённого анализа и выявленных требований к быстродействию и надёжности утверждён следующий стек технологий для реализации выпускной квалификационной работы:

- язык программирования: Python (с использованием `asyncio`);
- веб-фреймворк: FastAPI (сервер приложений Uvicorn);
- СУБД: Redis (режим персистентности AOF);
- целевая платформа: ОС Astra Linux Special Edition[10].

Данный выбор обеспечивает оптимальный баланс между производительностью, скоростью разработки и надёжностью функционирования системы в условиях высокой нагрузки.

1.4 Обзор аналогичных решений и обоснование разработки собственного программного комплекса

В рамках анализа предметной области необходимо рассмотреть существующие на рынке инструменты, применяемые для создания, эксплуатации и управления имитационными сервисами (заглушками). Поскольку разрабатываемая система предназначена для управления инфраструктурой заглушек в процессе нагрузочного тестирования, к аналогам следует отнести программные продукты, обеспечивающие виртуализацию сервисов (Service Virtualization), мокирование (Mocking), а также серверы приложений, традиционно используемые для хостинга Java-артефактов.

Сравнительный анализ проводится по критериям, критически важным для обеспечения стабильности нагрузочного стенда и автоматизации процессов тестирования:

- архитектура и потребление ресурсов: способность работать в условиях ограниченных аппаратных мощностей при запуске десятков экземпляров заглушек;

- возможности управления процессами (Orchestration): наличие инструментов для удалённого запуска, остановки и обновления версий приложений-заглушек;

- функциональность управления трафиком: наличие встроенных механизмов ограничения нагрузки (Rate Limiting) и эмуляции сетевых задержек;

- совместимость и импортозамещение: возможность автономной работы в среде ОС Astra Linux без зависимости от зарубежных облачных сервисов и проприетарных лицензий.

1.4.1 Анализ решения WireMock

WireMock – один из наиболее распространённых инструментов с открытым исходным кодом для виртуализации HTTP-сервисов. Продукт позволяет создавать заглушки, настраивать правила сопоставления запросов (request matching) и формировать динамические ответы.

Согласно официальной документации [11], WireMock поддерживает гибкую настройку ответов через JSON API, имитацию задержек (network latency) и ошибок (fault injection). Он может работать в двух режимах: как библиотека, встраиваемая в код автотестов, или как отдельный процесс (Standalone).

Недостатки в контексте поставленной задачи:

- ресурсоёмкость: WireMock разработан на Java и выполняется в виртуальной машине JVM. Запуск отдельного экземпляра WireMock для каждого имитируемого микросервиса создаёт значительные накладные расходы на

оперативную память (RAM) и процессорное время (CPU). В условиях нагрузочного тестирования, когда требуется поднять 50–100 заглушек, суммарные затраты ресурсов на поддержание работы самих заглушек становятся неприемлемыми;

- отсутствие встроенной оркестрации: в бесплатной версии (Open Source) WireMock отсутствует централизованная панель управления распределённым кластером. Инструмент фокусируется на логике ответа, а не на эксплуатации сервера;

- ограничения Rate Limiting: базовый функционал позволяет имитировать задержки, однако полноценный алгоритм ограничения количества запросов в секунду (RPS) с отбрасыванием лишнего трафика требует написания кастомных расширений (extensions) на Java [11].

1.4.2 Анализ решения MockServer

MockServer – популярное решение для мокирования систем, взаимодействующих по протоколам HTTP или HTTPS. Продукт часто используется в контейнеризированных средах (Docker, Kubernetes) для интеграционного тестирования.

MockServer предоставляет мощный API для создания «ожиданий» (Expectations) и верификации запросов. Согласно документации [12], он поддерживает проксирование трафика (Port Forwarding), запись проходящих запросов для последующего анализа и генерацию ответов на основе шаблонов (Mustache, Velocity).

Недостатки в контексте поставленной задачи:

- монолитная архитектура управления: MockServer хранит состояния «ожиданий» в памяти конкретного экземпляра приложения. Для создания распределённой системы требуется сложная настройка кластеризации, что утяжеляет инфраструктуру тестового стенда;

— сложность динамического управления: MockServer ориентирован на программное управление через API во время прогона тестов. Использование его как постоянно действующего инфраструктурного компонента требует разработки дополнительной обвязки для мониторинга его состояния и автоматического перезапуска (Self-healing) [12];

— зависимость от Java: аналогично WireMock, продукт написан на Java, что влечёт проблемы с «холодным стартом» и высоким потреблением памяти (Heap Size), снижающим плотность размещения заглушек на сервере.

1.4.3 Анализ решения Mountebank

Mountebank – инструмент с открытым исходным кодом, разработанный на платформе Node.js. Его ключевой особенностью является мультипротокольность: помимо HTTP/HTTPS, он поддерживает TCP, SMTP и другие протоколы через систему плагинов.

Mountebank использует концепцию «самозванцев» (imposters) – лёгких сервисов, слушающих определённые порты. Документация [13] указывает на возможность использования JavaScript-инъекций для реализации сложной логики обработки ответов.

Недостатки в контексте поставленной задачи:

— проблемы безопасности: возможность исполнения произвольного JS-кода (injection) в теле запроса создаёт уязвимости в контуре нагрузочного тестирования;

— отсутствие управления артефактами: Mountebank позволяет создать логику ответа (скрипт), но не предназначен для запуска и управления бинарными файлами (заглушками в виде .jar), предоставленными разработчиками;

— производительность в однопоточном режиме: в стандартной конфигурации Mountebank работает в одном потоке (Event Loop). При высокой нагрузке (тысячи RPS), характерной для нагрузочного тестирования, сложные

вычисления внутри одной заглушки могут заблокировать обработку запросов других заглушек, работающих в том же экземпляре Mountebank [13].

1.4.4 Анализ решения Apache Tomcat

Apache Tomcat – свободно распространяемый контейнер сервлетов, реализующий спецификации Jakarta Servlet и Jakarta Pages. Это классическое решение для хостинга Java-приложений, широко применяемое в промышленной эксплуатации.

Согласно официальной документации [14], Tomcat предоставляет развитые средства управления жизненным циклом веб-приложений. Компонент Manager App позволяет развёртывать (deploy), останавливать и перезапускать приложения без остановки сервера. Поддерживается кластеризация для репликации сессий [15]. Для защиты от перегрузок предусмотрен фильтр RateLimitFilter, позволяющий ограничивать количество запросов с одного IP-адреса [16].

Недостатки в контексте поставленной задачи:

- избыточность архитектуры (Overhead): Tomcat является полноценным сервлет-контейнером. Использование его для запуска примитивных заглушек нерационально с точки зрения потребления ресурсов. Запуск отдельного экземпляра Tomcat для каждой заглушки потребляет сотни мегабайт оперативной памяти только на нужды самого сервера, что критично при дефиците ресурсов;

- требования к формату артефактов: Tomcat работает с WAR-архивами. Однако современные микросервисы (и заглушки для них) чаще всего поставляются в виде Fat JAR (со встроенным сервером, например, Netty или Undertow). Для запуска таких заглушек в Tomcat требуется их пересборка и модификация кода, что усложняет процесс;

- статичность конфигурации ограничений: механизмы Rate Limiting в Tomcat конфигурируются декларативно через XML-файлы при старте. В за-

дачах нагрузочного тестирования требуется динамическое изменение параметров дросселирования трафика «на лету» (runtime) без перезагрузки сервера, что в Tomcat не реализовано «из коробки».

1.4.5 Сравнительная характеристика

Для наглядного сравнения рассмотренных решений с проектируемой системой составлена таблица 1.

Таблица 1 – Сравнительный анализ средств управления имитационными сервисами

Критерий	WireMock	Mock-Server	Mountebank	Apache Tomcat	Разраб. система
Управление JAR/WAR без пересборки	Нет	Нет	Нет	Частично	Да
Централизованная оркестрация	Нет	Нет	Нет	Нет	Да
Динамический Rate Limiting	Нет	Нет	Нет	Нет	Да
Потребление ресурсов	Высокое (JVM)	Высокое (JVM)	Среднее	Высокое (JVM)	Низкое
Совместимость с Astra Linux	Частично	Частично	Да	Частично	Да

1.4.6 Обоснование разработки собственного программного комплекса

Проведённый анализ показывает, что существующие на рынке решения делятся на две категории:

— инструменты логической виртуализации (WireMock, MockServer, Mountebank) – отлично справляются с задачей «что ответить на запрос»,

но не решают задачу управления инфраструктурой (запуск, перезапуск, обновление версий приложений);

— серверы приложений (Apache Tomcat) – умеют управлять жизненным циклом, но слишком тяжеловесны, требуют специфического формата упаковки (WAR) и сложны в динамической настройке сетевых параметров.

Ни одно из рассмотренных решений не предоставляет функционала «лёгковесного агента» для управления сторонними Java-процессами «как есть» (без перепакетки).

Разработка собственной автоматизированной информационной системы обоснована следующими факторами.

Специфика задачи (Lifecycle Management): разрабатываемая система решает задачу оркестрации процессов операционной системы. Она позволяет загружать бинарный файл заглушки (JAR), полученный от разработчиков, и управлять его исполнением на удалённом сервере. Это исключает трудозатраты на пересборку заглушек под WireMock или Tomcat.

Эффективность использования ресурсов: перенос логики управления на Python и использование асинхронного фреймворка FastAPI позволяет минимизировать накладные расходы самого управляющего контура. Система работает поверх заглушек, не внедряясь в их память, в отличие от тяжёлых Java-контейнеров.

Централизация и удобство эксплуатации: реализация архитектуры «Master-Agent» объединяет разрозненные серверы в единый кластер. Единая панель управления (Dashboard) позволяет инженеру тестирования массово обновлять версии заглушек и менять настройки лимитов (RPS) в реальном времени, что невозможно при использовании Standalone-решений.

Технологическая независимость и импортозамещение: система разрабатывается на базе открытых технологий, адаптирована для работы в ОС Astra Linux и не зависит от внешних репозиторий (Maven Central, Docker Hub) или лицензионных ключей, что гарантирует стабильность работы в закрытых корпоративных контурах РФ.

1.4.7 Вывод по подразделу

Разработка собственного программного комплекса является единственным целесообразным решением, закрывающим пробел между инструментами логического мокирования и средствами управления инфраструктурой, обеспечивая высокую производительность и гибкость конфигурации тестовых сред.

1.5 Формирование функциональных и технических требований к системе

На основании проведённого анализа предметной области и специфики задач нагрузочного тестирования сформированы требования к разрабатываемому программному комплексу. Система классифицируется как инструмент автоматизации инфраструктуры нагрузочного тестирования. Архитектура решения должна быть гибкой, допускающей работу как в распределённом контуре, так и в рамках одного физического или виртуального сервера.

1.5.1 Требования к режимам функционирования

Система должна поддерживать два сценария развёртывания без изменения кодовой базы:

- распределённый режим (Cluster Mode): классическая клиент-серверная архитектура, где управляющий узел (Master) администрирует множество удалённых узлов (Agents), на которых выполняются заглушки. Взаимодействие осуществляется по сети;

- автономный режим (Standalone Mode): консолидированный запуск всех компонентов системы на одном сервере. В этом режиме управляющий модуль и модуль исполнения заглушек функционируют в пределах одной операционной системы, обеспечивая изоляцию контура тестирования без необходимости сетевого взаимодействия между узлами инфраструктуры.

1.5.2 Функциональные требования

Функциональные требования определяют перечень задач, которые система должна выполнять для обеспечения процессов эмуляции сервисов.

1.5.2.1 Требования к подсистеме управления жизненным циклом (Runtime)

- поддержка форматов артефактов: система должна обеспечивать запуск Java-приложений, поставляемых в форматах исполняемых файлов `.jar` и веб-архивов `.war`;
- управление процессами: обеспечение запуска, корректной остановки и принудительного завершения процессов заглушек;
- конфигурация запуска: возможность задания аргументов виртуальной машины (JVM Options), включая параметры памяти (`-Xms`, `-Xmx`) и системные свойства, через графический интерфейс управляющего узла (Master) перед стартом заглушки;
- мониторинг: отслеживание статуса процесса (PID) и доступности его сетевого порта;
- работа с логами: сбор стандартных потоков вывода (`stdout`, `stderr`) запущенных процессов и их трансляция в интерфейс системы для отладки.

1.5.2.2 Требования к подсистеме проксирования запросов (Proxy Engine)

Система должна работать как прозрачный прокси-сервер (Transparent Proxy) на прикладном уровне, реализуя следующий алгоритм:

- а) приём входящего HTTP-запроса от внешней системы;
- б) извлечение метаданных: HTTP-метод, заголовки, тело запроса и целевой путь (URI);

в) выполнение нового запроса к целевой заглушке (на её локальный порт) с использованием извлечённого метода, заголовков и тела;

г) получение ответа от заглушки;

д) возврат полученного ответа инициатору запроса в неизменном виде.

Требования к управлению нагрузкой (Rate Limiting):

— наличие механизма ограничения количества запросов в секунду (RPS) для конкретной заглушки;

— активация лимита производится через вызов конфигурационного энд-поинта системы (API);

— при превышении лимита система должна возвращать HTTP-код 429 (Too Many Requests), не передавая запрос заглушке.

1.5.2.3 Требования к управляющему модулю (Master/Dashboard)

— управление артефактами: загрузка бинарных файлов (.jar, .war) в хранилище системы через веб-интерфейс;

— реестр сервисов: отображение списка всех загруженных заглушек, их текущего статуса («Запущен», «Остановлен», «Ошибка») и адресов;

— интерфейс конфигурации: предоставление полей ввода для настройки параметров запуска (JVM args) для каждого сервиса.

1.5.3 Нефункциональные (технические) требования

1.5.3.1 Требования к производительности

— вносимая задержка (Overhead): накладные расходы на проксирование (время обработки запроса внутри системы без учёта времени работы самой заглушки) не должны превышать 20 мс;

— пропускная способность: система должна обеспечивать высокую производительность проксирующего слоя. Снижение максимальной пропускной

способности (RPS) заглушки при работе через систему не должно превышать 10% относительно прямого обращения к заглушке.

1.5.3.2 Требования к надёжности

— сохранение конфигурации и восстановление работоспособности: при перезапуске системы (или сервера) должны сохраняться настройки запуска заглушек и ссылки на загруженные артефакты (Persistence); для заглушек, находившихся в активном состоянии до перезапуска, система должна предпринять автоматическую попытку повторного запуска процесса;

— изоляция сбоев: ошибка в одной заглушке не должна влиять на работоспособность остальных запущенных сервисов и самой управляющей системы.

1.5.4 Требования к программно-аппаратному обеспечению

1.5.4.1 Системные требования

— операционная система: серверная часть должна функционировать под управлением ОС Astra Linux Special Edition (версия 1.7 и выше) или аналогичных Linux-дистрибутивов;

— среда исполнения Java: поддержка OpenJDK 17[17] для запуска пользовательских артефактов.

1.5.4.2 Стек разработки

— язык программирования: Python (версия 3.10+) с использованием асинхронного подхода;

— веб-фреймворк: FastAPI (для реализации высокопроизводительного ввода-вывода);

— СУБД: Redis (для хранения конфигураций и реализации счётчиков Rate Limiter).

1.5.5 Требования к информационной безопасности

— контроль доступа: доступ к административному интерфейсу и API управления должен быть ограничен;

— безопасность данных: загружаемые исполняемые файлы должны храниться в изолированных директориях с правами доступа, исключающими их несанкционированную модификацию третьими лицами.

1.6 Выводы по первой главе

В первой главе выпускной квалификационной работы был проведён комплексный анализ предметной области и обоснована необходимость создания специализированного инструментария для автоматизации нагрузочного тестирования.

В ходе исследования выявлено, что использование традиционных серверов приложений и существующих аналогов (WireMock, MockServer, Mountebank) для управления распределённой сетью заглушек сопряжено с избыточным потреблением ресурсов и сложностями в эксплуатации, а также не удовлетворяет требованиям по импортозамещению.

На основании сравнительного анализа обоснован выбор технологического стека: язык программирования Python с использованием асинхронного подхода, веб-фреймворк FastAPI для обеспечения высокой пропускной способности и In-Memory СУБД Redis для хранения состояний с минимальной задержкой.

Определена монолитная архитектура будущего решения с управлением удалёнными хостами через asyncssh/SFTP и сформированы функциональные и технические требования к системе, включая поддержку форматов .jar и .war, механизмы ограничения нагрузки (Rate Limiting) и совместимость с

ОС Astra Linux. Полученные результаты служат базисом для проектирования и технической реализации системы во второй главе.

2 ПРОЕКТИРОВАНИЕ И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ УПРАВЛЕНИЯ

2.1 Разработка архитектуры распределённого программного комплекса

2.1.1 Выбор архитектурного стиля и топология развёртывания

Для разрабатываемого комплекса рассматривались два архитектурных стиля: микросервисный и монолитный. Микросервисная архитектура неоправданно усложнила бы систему: потребовала бы оркестратора контейнеров (внешняя зависимость, противоречащая требованию технологической независимости), ввела сетевые задержки между внутренними компонентами и увеличила операционную нагрузку при разработке одним специалистом. Разрабатываемая система является «тонким» координирующим слоем над имитационными сервисами, поэтому накладные расходы микросервисной декомпозиции несоразмерны её выгодам.

Выбрана монолитная архитектура с явно выраженной модульной структурой. Сравнительный анализ подходов приведён в таблице 2.

Таблица 2 – Сравнение архитектурных подходов

Критерий	Монолитная архитектура	Микросервисная архитектура
Накладные расходы	Минимальные	Высокие
Сложность развёртывания	Низкая	Высокая
Зависимость от внешних систем	Отсутствует	Требуется оркестратор
Сопровождаемость	Высокая	Низкая

Критерий	Монолитная архитектура	Микросервисная архитектура
Масштабирование компонентов	Не поддерживается	Поддерживается

Распределённость системы реализуется на уровне топологии развёртывания, а не внутренней декомпозиции: единый управляющий процесс координирует целевые хосты через реестр. Каждый хост реестра описывается сетевым адресом, учётной записью SSH и рабочей директорией. При регистрации заглушки оператор выбирает целевой хост, что позволяет совмещать локальное (127.0.0.1) и удалённое (asyncssh/SFTP) размещение в рамках одной инфраструктуры без изменения конфигурации приложения. Компонент Host Resolver предоставляет функцию `is_local(hostname)`, инкапсулируя точку ветвления между локальными системными вызовами и SSH-операциями.

2.1.2 Архитектура системы в нотации C4 Model

Для формализованного описания архитектуры применяется нотация C4 Model[18]. Контекстная диаграмма (рисунок 1) представляет систему в окружении внешних акторов: инженера тестирования (управляет инфраструктурой через веб-интерфейс), потребителя (HTTP-клиент: нагрузочный инструмент или тестируемая система), Prometheus (сбор метрик) и целевого хоста Astra Linux[10] с Java-процессами[17].

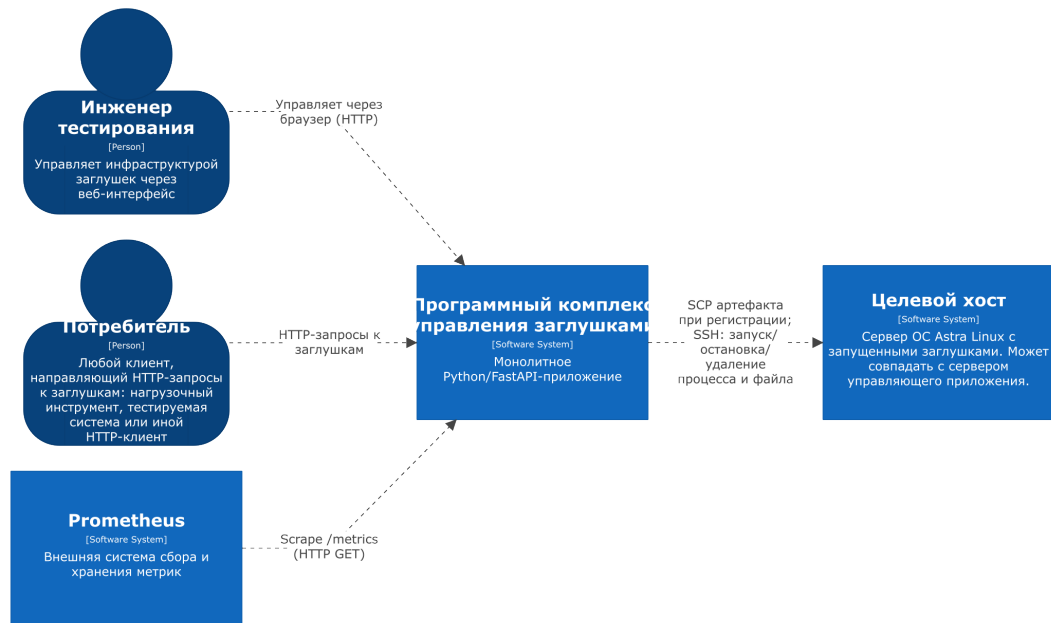


Рисунок 1 – Контекстная диаграмма системы (C4 Level 1)

Контейнерная диаграмма (рисунок 2) раскрывает три программных контейнера системы (в терминологии C4 Model – логически обособленные исполняемые единицы). *Веб-интерфейс* (FastAPI / Jinja2 SSR[19]) формирует HTML-страницы на стороне сервера без зависимостей от внешних CDN. *REST API* (Python / FastAPI / Uvicorn) реализует всю бизнес-логику. *Redis* (AOF-персистентность) хранит конфигурации и счётчики Rate Limiter. Выделенное хранилище артефактов намеренно отсутствует: бинарный файл заглушки принимается во временный каталог, немедленно копируется на целевой хост по SFTP и удаляется.

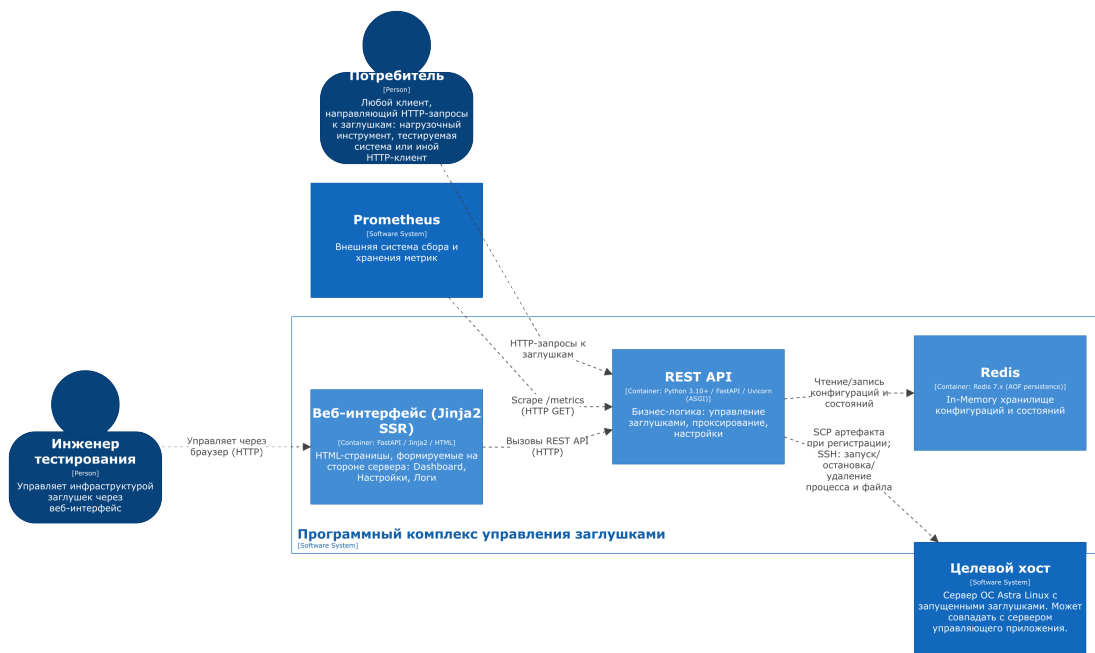


Рисунок 2 – Контейнерная диаграмма системы (C4 Level 2)

Компонентная диаграмма (рисунок 3) детализирует внутреннюю структуру REST API. Назначение шести компонентов приведено в таблице 3.

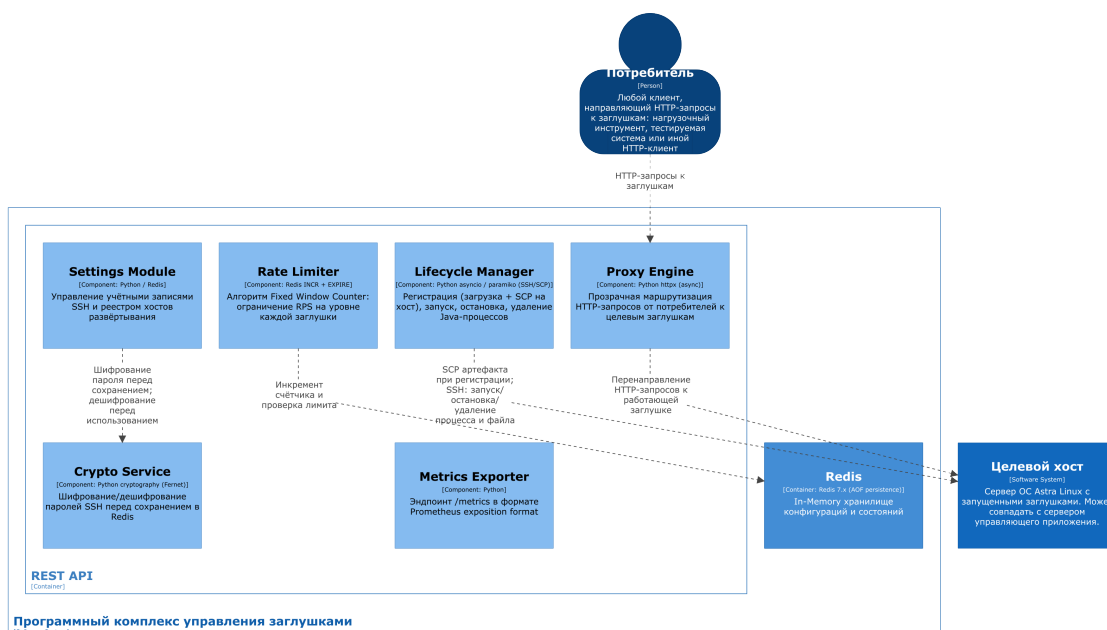


Рисунок 3 – Компонентная диаграмма REST API (C4 Level 3)

Таблица 3 – Компоненты REST API

Компонент	Назначение
Lifecycle Manager	Регистрация (SFTP-загрузка на хост[20]), запуск (nohup через SSH), остановка и удаление Java-процессов
Proxy Engine	Прозрачная маршрутизация HTTP-запросов к заглушкам через httpx[21] с сохранением метода, заголовков и тела
Rate Limiter	Алгоритм Fixed Window Counter на уровне каждой заглушки; управляется флагом <code>rate_limit_enabled</code> без перезапуска
Settings Module	Управление учётными записями SSH и реестром хостов развёртывания (см. раздел 2.1.4)
Crypto Service	Шифрование/дешифрование паролей SSH алгоритмом AES-128 (Fernet[22]); ключ хранится в файле с правами 600
Metrics Exporter	Эндпоинт <code>/metrics</code> в формате Prometheus exposition format[23][24]; детали в разделе 2.6

2.1.3 Диаграмма развёртывания

Диаграмма развёртывания (рисунок 4) описывает физическое размещение компонентов. Сервер управляющего приложения содержит FastAPI-процесс (Uvicorn) и Redis. Бинарные файлы заглушек постоянно хранятся только на целевых хостах в рабочих директориях; временный каталог на сервере используется исключительно на время одной SFTP-операции. На целевых хостах не требуется установки каких-либо компонентов системы – необходимы лишь SSH-сервер, учётная запись с правами на запись и исполняемый файл Java.

КАРТИНКА - ЗАГЛУШКА

Рисунок 4 – Диаграмма развёртывания

2.1.4 Архитектура модуля настроек и реестр хостов

Модуль настроек (Settings Module) управляет двумя категориями конфигурационных данных. **Учётные записи ОС Linux** используются для SSH-аутентификации на целевых хостах; применение учётных записей с ограниченными правами минимизирует риски при нештатном поведении Java-процессов. Пароли хранятся в Redis в зашифрованном виде: Crypto Service применяет AES-128 (Fernet[22]), ключ хранится вне Redis в файле с правами 600; дешифрование выполняется только в оперативной памяти на время установки соединения.

Реестр хостов представляет собой список серверов для развёртывания заглушек. Параметры записи приведены в таблице 4.

Таблица 4 – Параметры модуля настроек системы

Параметр	Тип	Описание
hostname	String	IP-адрес или имя хоста; уникальный идентификатор
ssh_port	Integer	Порт SSH (по умолчанию 22)

Параметр	Тип	Описание
account_uuid	UUID	ID учётной записи SSH из реестра accounts
working_dir	String	Абсолютный путь рабочей директории на хосте
java_path	String	Путь к исполняемому файлу Java (явный, т.к. JAVA_HOME в корп. средах нередко не задан)
mock_port_min	Integer	Нижняя граница диапазона TCP-портов для заглушек
mock_port_max	Integer	Верхняя граница диапазона TCP-портов для заглушек
description	String	Текстовое описание хоста

Диапазон `mock_port_min/mock_port_max` обеспечивает совместимость с политиками межсетевого экрана в закрытых корпоративных сетях, где разрешается трафик только на заранее согласованные порты.

2.1.5 Типовые сценарии взаимодействия компонентов

В таблице 5 приведены пять типовых сценариев, иллюстрирующих сквозное взаимодействие компонентов. Ключевое архитектурное решение: бинарный артефакт копируется на целевой хост однократно при регистрации; все последующие операции работают с файлом на хосте.

Таблица 5 – Типовые сценарии взаимодействия компонентов

Сценарий	Последовательность действий
Регистрация заглушки	Загрузка файла → временный каталог на сервере → SFTP на целевой хост → удаление временного файла → <code>HSET mocks:{filename} + SADD mocks:registry</code> (один Pipeline), статус REGISTERED

Сценарий	Последовательность действий
Запуск заглушки	Считать конфигурацию из Redis → дешифровать пароль → определить свободный порт через SSH → <code>nohup {java_path} {jvm_args} -jar {artifact_path} -server.port={port}</code> → сохранить PID, порт, статус RUNNING в Redis
Остановка заглушки	Считать PID из Redis → выполнить <code>kill {PID}</code> через SSH (SIGTERM; при отсутствии реакции – SIGKILL) → <code>HSET mocks:{filename} status STOPPED</code> , поля <code>pid</code> и <code>port</code> очищаются
Обработка запроса	Считать статус и флаг <code>rate_limit_enabled</code> из Redis → при статусе $\neq$ RUNNING: HTTP 503 → при включённом Rate Limiter: <code>INCR rate:{mock}:{window}</code> , при превышении: HTTP 429 → иначе: проксировать через <code>httpx</code> , вернуть ответ без изменений
Переключение Rate Limiter	<code>PATCH /api/v1/mocks/{name}/rate-limit</code> → атомарный <code>HSET rate_limit_enabled</code> в Hash-записи заглушки → применяется со следующего запроса без перезапуска
Удаление заглушки	Остановить процесс через SSH (SIGTERM / SIGKILL) → <code>SSH rm {artifact_path}</code> → <code>DEL mocks:{filename} + SREM mocks:registry</code>

2.2 Проектирование структур данных для хранения конфигураций и состояний в Redis

2.2.1 Обоснование модели данных

Redis выполняет роль единственного хранилища всех оперативных данных системы: конфигураций заглушек, параметров хостов, учётных записей, состояний процессов, счётчиков Rate Limiter и реестров объектов. Бинарные файлы артефактов в Redis не хранятся – единственной постоянной копией является файл на целевом хосте. Выбор Redis обусловлен временем доступа менее 1 мс, критически важным для Rate Limiter, выполняемого при обработке каждого входящего запроса.

Для хранения конфигурационных объектов (заглушки, хосты, учётные записи, глобальные настройки) используется тип Hash: каждый атрибут объекта хранится как отдельное поле, что позволяет обновлять конкретные поля командой HSET без перезаписи всей записи и читать объект целиком командой HGETALL за одну операцию. Для реестров идентификаторов применяется тип Set, гарантирующий уникальность. Для счётчиков Rate Limiter – тип String с атомарными командами INCR и EXPIRE.

В таблице 6 приведена сводная схема пространств имён ключей.

Таблица 6 – Пространства имён ключей Redis

Ключ	Тип	Назначение
mocks:{filename}	Hash	Конфигурация и состояние заглушки
mocks:registry	Set	Множество имён файлов всех заглушек
hosts:{hostname}	Hash	Конфигурация целевого хоста
hosts:registry	Set	Множество hostname всех хостов
accounts:{uuid}	Hash	Учётная запись SSH

Ключ	Тип	Назначение
accounts:registry	Set	Множество UUID учётных записей
rate:{filename}:{window}	String	Счётчик Rate Limiter (с TTL)
settings:global	Hash	Глобальные параметры приложения

2.2.2 Структуры данных объектов системы

Структура Hash-записи заглушки (`mocks:{filename}`) приведена в листинге 2.1. Имя файла является уникальным идентификатором заглушки: при попытке зарегистрировать файл с уже существующим именем система возвращает ошибку. Поле `artifact_path` вычисляется однократно при регистрации как конкатенация `working_dir` хоста и имени файла, что исключает повторное обращение к записи хоста при каждой операции. Поля `port`, `pid` и `started_at` заполняются после успешного запуска и сбрасываются при остановке.

Листинг 2.1 – Структура Hash-записи заглушки в Redis

```

1 HSET mocks:{filename}
2     hostname          test-server-01
3     port              8105
4     pid               24731
5     status            RUNNING
6     jvm_args           -Xms128m -Xmx256m
7     rate_limit         500
8     rate_limit_enabled true
9     registered_at      2024-11-01T10:30:00+00:00
10    started_at         2024-11-01T10:35:00+00:00
11    artifact_path
    ↪ /opt/mock-services/payment-service-stub.jar

```

Таблица 7 – Поля Hash-записи заглушки

Поле	Тип	Описание
hostname	String	Хост развёртывания заглушки
port	Integer/null	TCP-порт процесса
pid	Integer/null	PID процесса на хосте
status	String	REGISTERED / RUNNING / STOPPED / ERROR
jvm_args	String	Аргументы JVM
rate_limit	Integer	Макс. запросов в секунду
rate_limit_enabled	Boolean	Флаг активности Rate Limiter
registered_at	ISO 8601	Дата регистрации
started_at	ISO 8601/null	Дата последнего запуска
artifact_path	String	Абсолютный путь к JAR/WAR на целевом хосте

Структура Hash-записи хоста (`hosts:{hostname}`) приведена в листинге 2.2. Hostname одновременно является уникальным идентификатором записи и адресом для asyncssh/SFTP-подключения, обеспечивая прямой доступ за одну операцию HGETALL. Поле `status` отражает результат последней фоновой проверки SSH-доступности (`AVAILABLE / UNAVAILABLE / UNKNOWN`); поле `account_uuid` указывает на запись `accounts:{uuid}` с зашифрованными учётными данными.

Листинг 2.2 – Структура Hash-записи хоста в Redis

```
1 HSET hosts:{hostname}
2     ssh_port      22
3     account_uuid  e5f6a7b8-c9d0-1234-efab-567890abcdef
4     working_dir   /opt/mock-services
5     java_path     /usr/lib/jvm/java-17-openjdk-amd64/bin/java
6     mock_port_min 8100
7     mock_port_max 8200
8     description   Стенд нагрузочного тестирования – контур
                    ↪ препром.
9     status        AVAILABLE
10    last_checked_at 2024-11-01T10:29:55+00:00
```

Структура Hash-записи учётной записи (`accounts:{uuid}`) приведена в листинге 2.3. Поле `password_enc` содержит Base64-кодированный зашифрованный блок (Fernet/AES-128); дешифрование выполняется Crypto Service только в оперативной памяти на время установки SSH-соединения – расшифрованный пароль никогда не записывается на диск.

Листинг 2.3 – Структура Hash-записи учётной записи в Redis

```
1 HSET accounts:{uuid}
2     username      mock-runner
3     password_enc   gAAAAAB1...base64-encoded-ciphertext...
4     description    Сервисный пользователь для запуска заглушек
5     created_at     2024-10-15T09:00:00+00:00
```

ER-диаграмма связей объектов в Redis приведена на рисунке 5.



Рисунок 5 – ER-диаграмма объектов в Redis

2.2.3 Счётчик Rate Limiter и глобальные настройки

Для каждой заглушки и каждого временного окна в Redis хранится счётчик по ключу `rate:{filename}:{window}`, где `{window}` – целочисленное деление текущего Unix-timestamp на длину окна в секундах. Алгоритм обработки запроса описан в листинге 2.4.

Листинг 2.4 – Псевдокод алгоритма Fixed Window Counter

```
1 def check_rate_limit(filename: str, limit: int, window_size: int =  
    ↪ 1) -> bool:  
2     # Шаг 1: вычислить метку текущего окна  
3     window = floor(current_unix_timestamp() / window_size)  
4     key     = f"rate:{filename}:{window}"  
5     # Шаг 2: атомарный инкремент счётчика  
6     count = INCR(key)  
7     # Шаг 3: установить TTL при создании нового окна  
8     if count == 1:  
9         EXPIRE(key, window_size * 2) # TTL = 2 окна (запас на сдвиг  
            ↪ времени)  
10    # Шаг 4: проверить лимит  
11    if count > limit:  
12        return False # запрос отклонён -> HTTP 429  
13    return True      # запрос разрешён
```

TTL устанавливается равным удвоенному размеру окна и только при первом запросе (INCR возвращает 1), что гарантирует самоочистку устаревших ключей и предотвращает утечку памяти. Атомарность INCR[25] обеспечивает корректный подсчёт при конкурентном поступлении запросов без транзакций и блокировок.

Глобальные параметры приложения хранятся под ключом `settings:global` (Hash). Структура записи приведена в листинге 2.5, описание полей – в таблице 8.

Листинг 2.5 – Структура Hash-записи глобальных настроек в Redis

```
1 HSET settings:global  
2     rate_limit_window_size      1  
3     host_check_interval_sec     30  
4     proxy_timeout_sec           10  
5     log_retention_lines         1000
```

Таблица 8 – Поля Hash-записи глобальных настроек системы

Поле	Тип	Описание
rate_limit_window_size	Integer	Размер окна Rate Limiter (сек)
host_check_interval_sec	Integer	Интервал проверки доступности хостов (сек)
proxy_timeout_sec	Integer	Таймаут проксирования (сек)
log_retention_lines	Integer	Макс. строк лога процесса в буфере

2.2.4 Персистентность и стратегия восстановления

Redis настроен в режиме AOF[9] (`appendfsync everysec`): каждая операция записи фиксируется в журнале, синхронизируемом с диском раз в секунду; максимальная потеря данных при аварии – не более одной секунды.

При перезапуске управляющего приложения выполняется следующая процедура восстановления: считывается реестр заглушек (`SMEMBERS mocks:registry`), затем для каждой заглушки читается полная Hash-запись (`HGETALL`). Для каждой заглушки со статусом `RUNNING` выполняется SSH-проверка фактического наличия процесса по сохранённому PID. При подтверждении – статус сохраняется, восстанавливается HTTP-клиент прокси-движка. При отсутствии процесса – предпринимается автоматический перезапуск с сохранёнными JVM-аргументами; в случае успеха запись обновляется новыми `pid`, `port` и `started_at`, при неудаче – ошибка фиксируется в журнале, конфигурация остаётся неизменной для ручного вмешательства.

В таблице 9 приведена матрица операций Redis по компонентам системы.

Таблица 9 – Матрица операций Redis по компонентам системы

Компонент	Операции Redis	Назначение
Lifecycle Manager	HSET, HGETALL, HGET, SADD, SREM, DEL	Управление процессами
Proxy Engine	HGETALL, HMGET	Чтение данных заглушки
Rate Limiter	INCR, EXPIRE	Счётчик запросов
Settings Module	HSET, HGETALL, HGET, SADD, SREM, DEL	Управление конфигурациями
Metrics Exporter	SMEMBERS, HGETALL (Pipeline)	Сбор метрик

Для Metrics Exporter применяется конвейеризация команд Redis (Pipeline): сначала читается список ключей из реестра, затем все операции чтения отправляются единым пакетом, снижая суммарные сетевые задержки с $O(n)$ до фактически $O(1)$ по числу round-trip.

2.3 Разработка макетов приложения

2.3.1 Обоснование стиля и цветового решения

Целевая аудитория системы – инженеры тестирования, одновременно отслеживающие состояние десятков заглушек. Это определяет два ключевых требования: минимальную утомляемость при длительной работе и мгновенную читаемость статусов без анализа текстовых подписей.

Выбрана тёмная цветовая схема (dark theme) как стандарт для инструментов класса DevOps и Operations. Статусы объектов кодируются цветом однозначно: зелёный (RUNNING), жёлтый (REGISTERED / STOPPED), красный (ERROR / UNAVAILABLE). Акцентный цвет – синий #4F8EF7 с коэффициентом контрастности выше 4,5:1 (соответствует WCAG 2.1 уровня AA[26]).

Основной шрифт – Inter (входит в базовую поставку большинства дистрибутивов Linux, не требует внешних CDN), моноширинный – JetBrains Mono для технических идентификаторов. Компоновка страниц: постоянная боковая панель навигации шириной 220 px и основная рабочая область. Сводная характеристика визуальной системы приведена в таблице 10.

Таблица 10 – Визуальная система интерфейса

Элемент	Значение	Назначение
Фон контента	#121218	Основной фон
Фон боковой панели	#1A1A24	Навигация
Акцентный цвет	#4F8EF7	Кнопки, ссылки
Статус RUNNING	#22C55E	Процесс запущен
Статус REGISTERED / STOPPED	#EAB308	Зарегистрирован, не запущен
Статус ERROR / UNAVAILABLE	#EF4444	Ошибка/недоступен
Основной текст	#E2E8F0	Основной контент
Вторичный текст	#94A3B8	Метки, подсказки
Основной шрифт	Inter	Текст
Моноширинный шрифт	JetBrains Mono	Код, пути, аргументы

2.3.2 Структура навигации и макеты экранных форм

Все экранные формы разделяют единый шаблон-«обёртку»: постоянная боковая панель с тремя разделами (**Dashboard**, **Настройки**, **Журналы**) и верхняя строка с заголовком текущего раздела и индикатором подключения к Redis. Макет общего хрома приложения представлен на рисунке 6.

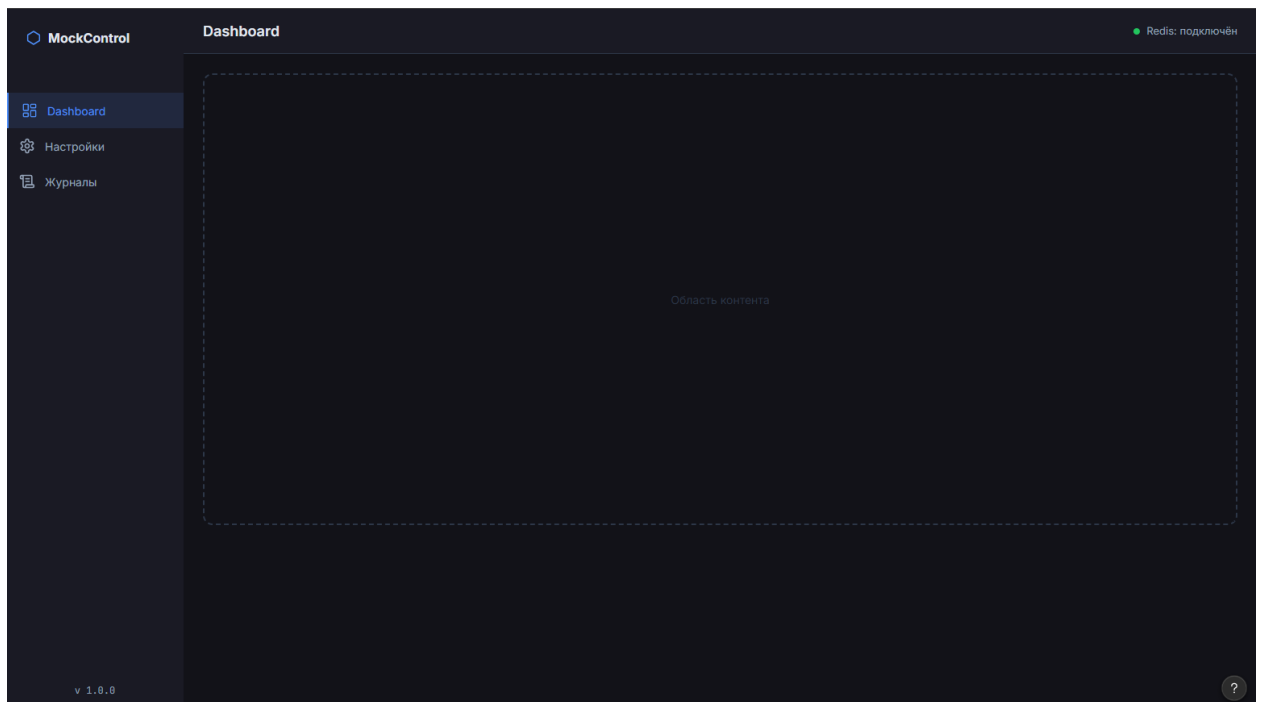


Рисунок 6 – Макет общего хрома приложения с боковой навигацией

Dashboard (рисунок 7) является основным рабочим экраном. Верхняя часть содержит четыре сводные карточки: всего заглушек, запущено, ошибок, хостов доступно. Таблица заглушек включает столбцы: имя файла (моноширинный шрифт), целевой хост, статусный бейдж, порт, тумблер Rate Limiter и кнопки действий. Кнопка «Зарегистрировать заглушку» открывает модальное окно.

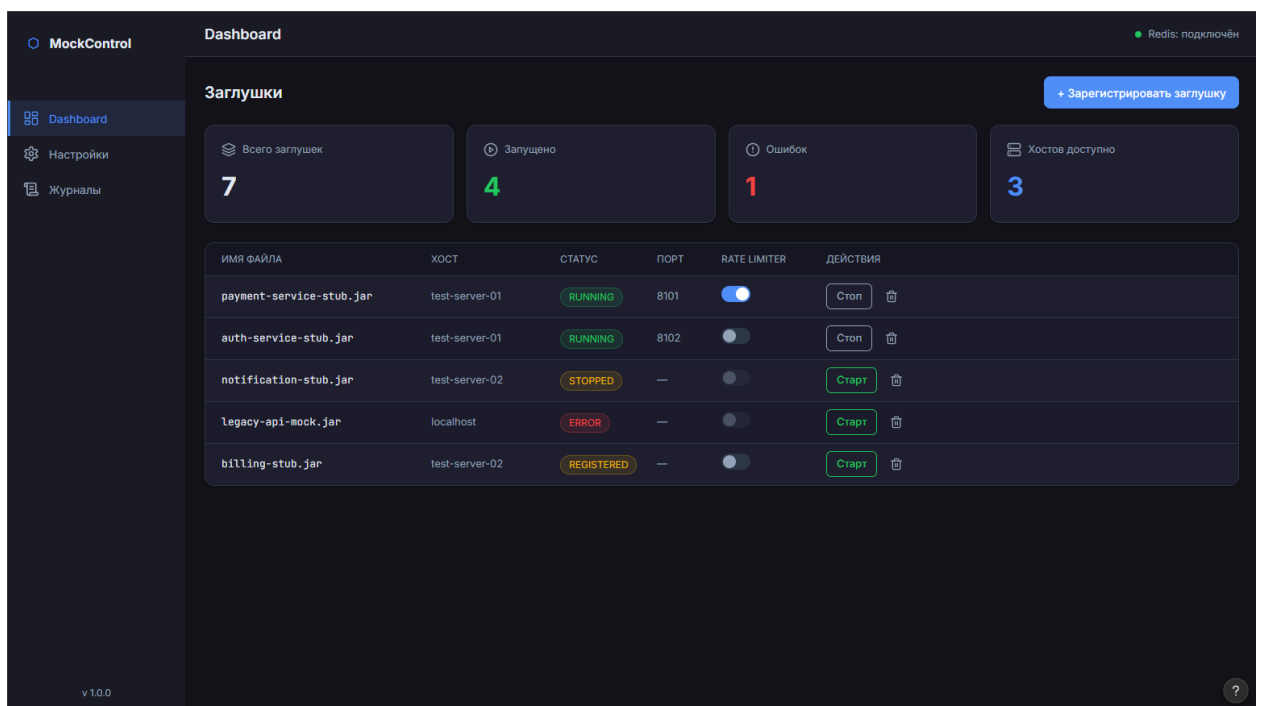


Рисунок 7 – Макет главной панели управления (Dashboard)

Модальное окно регистрации заглушки (рисунок 8) открывается поверх Dashboard без перехода на отдельную страницу, сохраняя контекст. Форма содержит: зону перетаскивания файла (.jar/.war), выпадающий список целевого хоста, поле JVM-аргументов (моноширинный шрифт), числовое поле лимита Rate Limiter и флажок немедленного запуска после регистрации.

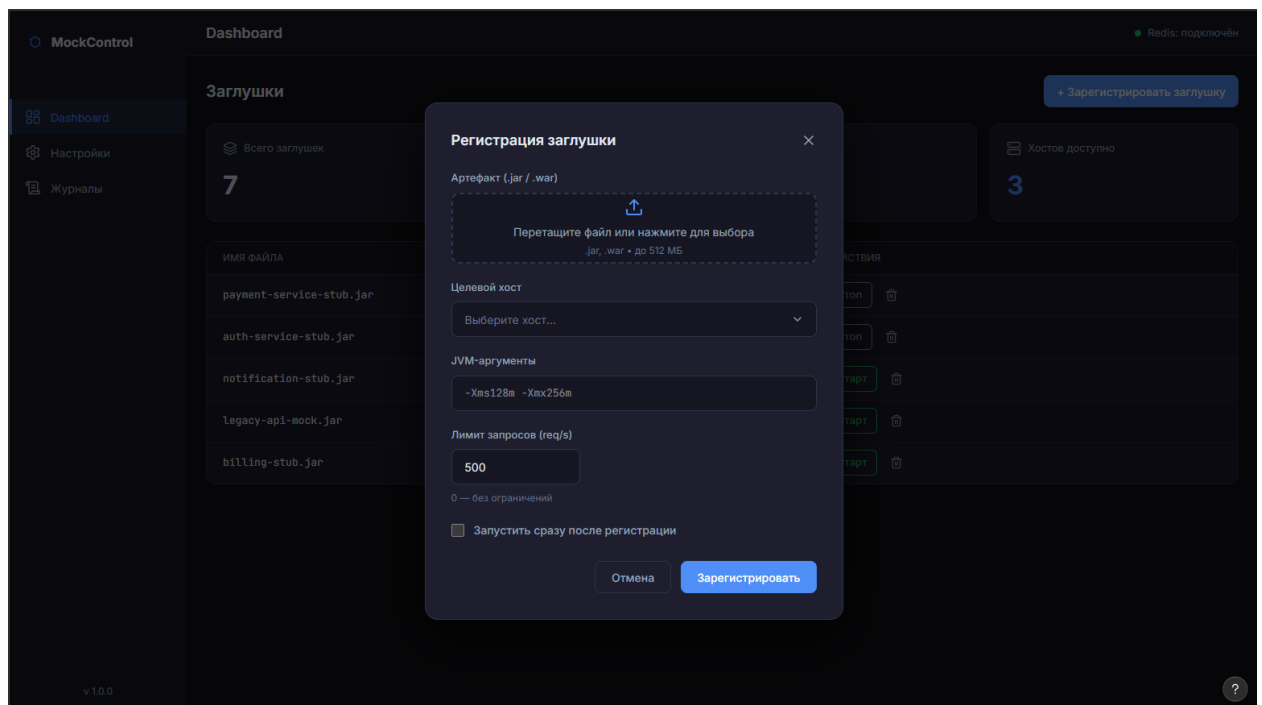


Рисунок 8 – Макет модального окна регистрации новой заглушки

Страница настроек (рисунок 9) организована в виде двух вкладок: «Хосты» и «Учётные записи». Вкладка «Хосты» отображает таблицу целевых серверов с индикатором SSH-доступности и временем последней проверки. Пароли учётных записей в таблице скрыты строкой «••••••••». Кнопка «Добавить хост» открывает модальную форму (рисунок 10) с полями реестра хостов из раздела 2.1.4.

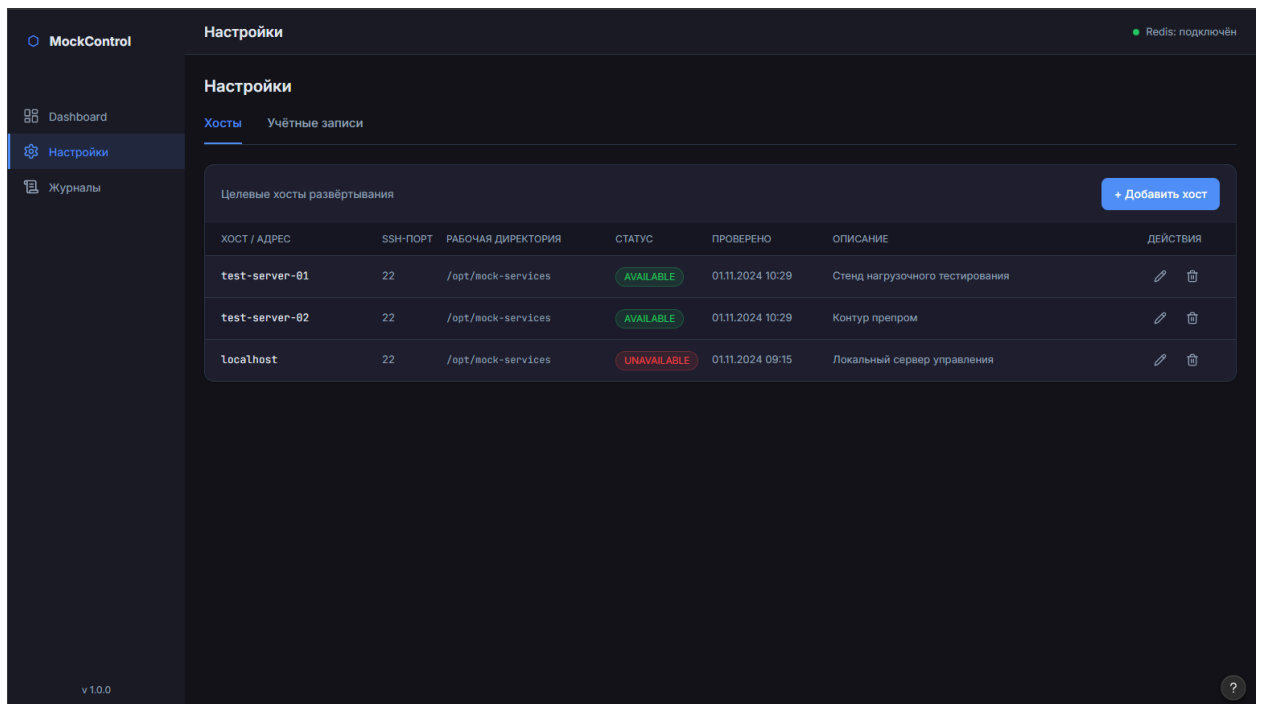


Рисунок 9 – Макет страницы настроек, вкладка «Хосты»

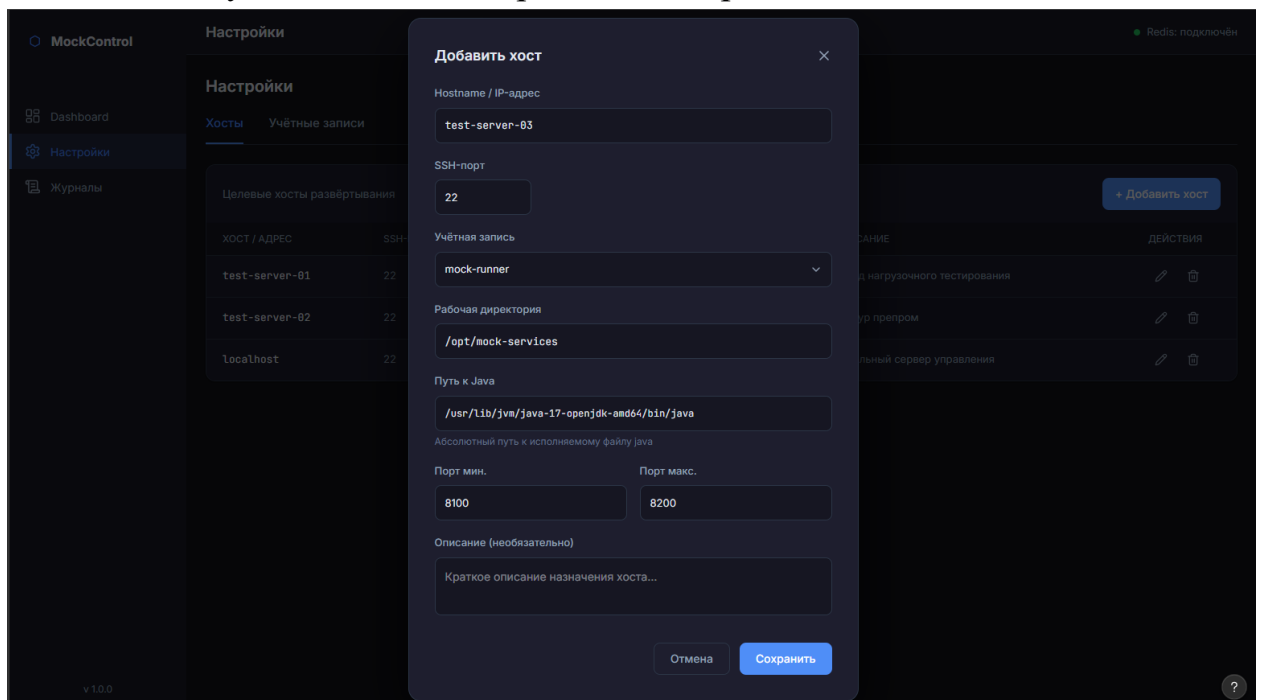


Рисунок 10 – Макет модального окна добавления нового хоста

Страница журналов (рисунок 11) предназначена для просмотра stdout/stderr Java-процессов. Выбор заглушки – через выпадающий список. Основную часть экрана занимает терминальный блок: строки с ERROR и WARN выделяются соответственно красным и жёлтым фоном. Панель управления содержит тумблер автообновления, кнопку ручного обновления и очистки буфера; строка состояния показывает PID процесса и назначенный порт.

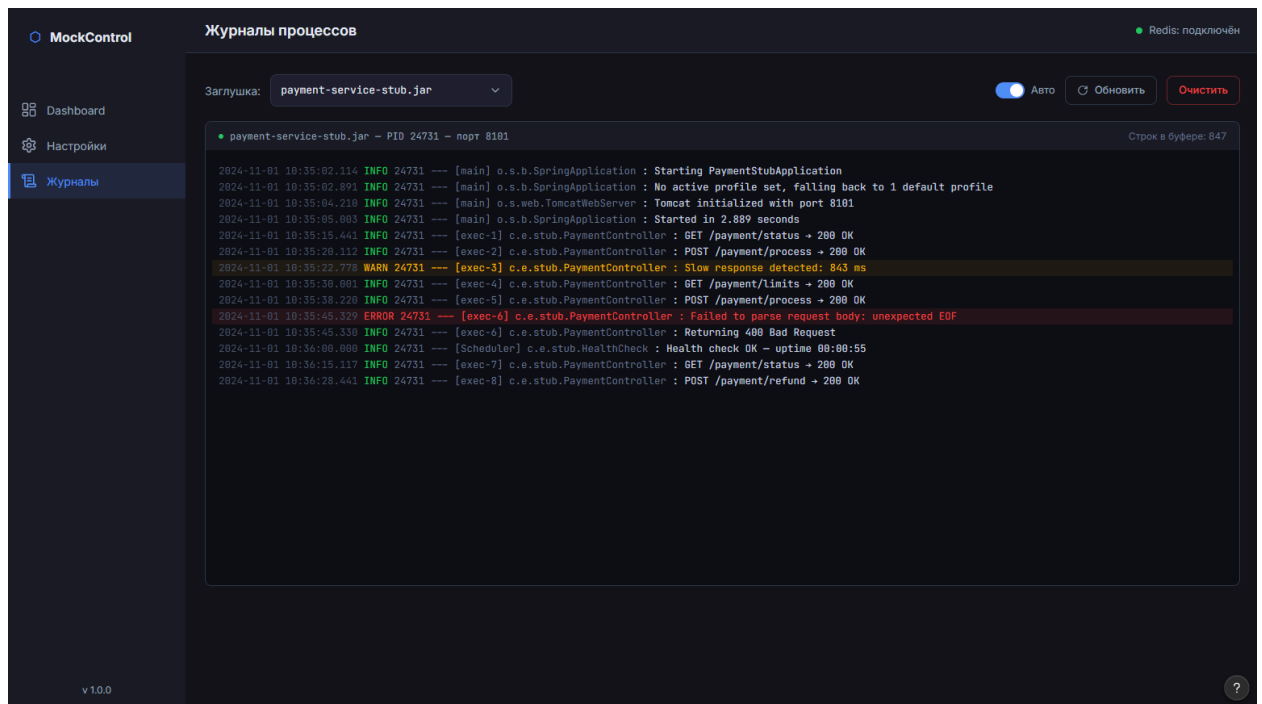


Рисунок 11 – Макет страницы просмотра журналов процессов

2.4 Реализация асинхронных механизмов управления процессами и ввода пользовательских параметров.

2.4.1 Архитектура асинхронного управления процессами

Все методы класса `LifecycleManager` реализованы как корутины Python (`async/await`), что позволяет событийному циклу `asyncio`[3] переключаться между операциями ввода-вывода без блокировки вычислительного потока. Операции управления ветвятся по типу целевого хоста: компонент `Host Resolver` предоставляет функцию `is_local(hostname)`, кэшированную через `functools.lru_cache`. Для локального хоста применяются встроенные средства Python и системные вызовы ОС; для удалённого – пул постоянных SSH-соединений (`SSHPool`) на базе `asyncssh`. Ветвление инкапсулировано внутри `LifecycleManager` и прозрачно для API-слоя.

2.4.2 Запуск Java-процесса

При запуске на **локальном хосте** поиск свободного TCP-порта выполняется в `asyncio.to_thread` (блокирующий `socket.bind` вынесен в пул потоков). JVM-аргументы из Redis разбираются функцией `shlex.split`, которая корректно обрабатывает кавычки и экранирование, — это позволяет задавать аргументы со встроенными пробелами: `Dmy.prop=value with spaces`. Процесс запускается через `asyncio.create_subprocess_exec` с параметром `start_new_session=True`, обеспечивающим независимость Java-процесса от группы процессов управляющего приложения. Реализация приведена в листинге 2.6.

Листинг 2.6 – Запуск Java-процесса на локальном хосте

```
1 async def start_mock(self, filename: str) -> None:
2     if self._is_local(hostname):
3         port = await self._pick_free_port_local(pmin, pmax)
4         log_file_path = _log_path_for_filename(filename)
5         # Открытие лог-файла вынесено в поток (I/O-bound операция)
6         lf = await asyncio.to_thread(open, log_file_path, "ab")
7         try:
8             argv = [java_path]
9             argv.extend(shlex.split(jvm_args) if jvm_args.strip()
10                ↪ else [])
11             argv.extend(["-jar", artifact_path, f"{{port}}"])
12             # Неблокирующий запуск дочернего процесса
13             proc = await asyncio.create_subprocess_exec(
14                 *argv,
15                 stdin=asyncio.subprocess.DEVNULL,
16                 stdout=lf,
17                 stderr=asyncio.subprocess.STDOUT,
18                 start_new_session=True)
19         finally:
20             lf.close()
21     pid = proc.pid
```

При запуске на **удалённом хосте** поиск свободного порта производится через SSH-команду `ss -tln`. Токены JVM-аргументов дополнительно экранируются `shlex.quote` перед включением в строку команды, что предотвращает инъекцию оболочки. Конструкция `nohup & echo $!` запускает процесс в фоновом режиме и возвращает его PID в stdout. Формирование команды приведено в листинге 2.7.

Листинг 2.7 – Формирование SSH-команды запуска Java-процесса на удалённом хосте

```
1 # Экранирование JVM-аргументов для безопасной передачи через SSH
2 jvm_part = "".join(
3     shlex.quote(t) for t in shlex.split(jvm_args.strip())
4 ) if jvm_args.strip() else ""
5
6 mid = f"{jvm_part} " if jvm_part else ""
7
8 # Итоговая команда: nohup, перенаправление вывода, вывод PID
9 cmd = (
10     f"nohup {shlex.quote(java_path)} {mid}"
11     f"-jar {shlex.quote(artifact_path)} {int(port)} "
12     f">> {shlex.quote(log_remote)} 2>&1 & echo $!"
13 )
14
15 proc = await self._ssh.run_command(
16     hostname, user, pwd, cmd, port=ssh_port, timeout=60.0
17 )
18
19 # Последняя строка вывода содержит PID запущенного процесса
20 pid = int(proc.stdout.strip().splitlines()[-1])
```

2.4.3 Остановка процесса, проверка живости и пул

SSH-соединений

Остановка реализует двухэтапный протокол: сначала отправляется SIGTERM, затем в течение 5 секунд (50 итераций `asyncio.sleep(0.1)`) проверяется живость процесса; при неуспехе – SIGKILL. Ожидание через `asyncio.sleep` не блокирует событийный цикл, позволяя серверу обраба-

тывать другие запросы. Проверка живости ветвится аналогично запуску: локально – `os.kill(pid, 0)` в `asyncio.to_thread`, удалённо – `kill -0 {pid}` через SSH. Реализация обоих методов приведена в листинге 2.8.

Листинг 2.8 – Методы проверки живости процесса

```
1 # Локальная проверка: системный вызов в пуле потоков
2 async def _process_alive_local(self, pid: int) -> bool:
3     def check() -> bool:
4         try:
5             os.kill(pid, 0)    # сигнал 0: только проверка
6                               → существования
7         except OSError:
8             return False      # процесс не найден или нет прав
9         return True
10
11     return await asyncio.to_thread(check)
12
13 # Удалённая проверка: команда kill -0 через SSH
14 async def _process_alive_remote(
15     self, hostname: str, username: str, password: str,
16     ssh_port: int, pid: int,
17 ) -> bool:
18     proc = await self._ssh.run_command(
19         hostname, username, password,
20         f"kill -0 {int(pid)} 2>/dev/null && echo OK || echo NO",
21         port=ssh_port, timeout=15.0,
22     )
23     return (proc.stdout or "").strip().endswith("OK")
```

SSHPool хранит по одному постоянному соединению на хост, защищённому `asyncio.Lock` от гонок при переподключении. Перед каждым использованием соединение проверяется командой `true`; при неуспехе – пересоздаётся прозрачно для вызывающего кода. Передача артефактов реализована через `asyncssh.start_sftp_client` поверх существующего соединения, что исключает дополнительные SSH-сессии.

2.4.4 Восстановление состояния при перезапуске

При старте приложения метод `restore_state` обходит реестр заглушек и для каждой со статусом `RUNNING` проверяет фактическое наличие процесса на хосте. При подтверждении восстанавливается HTTP-клиент прокси-движка. При отсутствии процесса (или при повреждённых полях `pid/port`) система автоматически инициирует перезапуск: вызывается `_start_mock_from_data`, запись в Redis обновляется новыми `pid`, `port` и `started_at`. При неудаче ошибка фиксируется в журнале. Реализована стратегия «наилучшего усилия» (`best-effort restart`), минимизирующая ручное вмешательство оператора.

2.5 Разработка модуля динамического проксирования запросов на базе FastAPI и алгоритма Rate Limiting

2.5.1 Архитектура прокси-модуля и реестр HTTP-клиентов

Модуль реализует прозрачный обратный прокси: входящий запрос к заглушке перенаправляется к её Java-процессу, ответ возвращается клиенту без изменений. Прокси-роутер подключён к FastAPI последним – после всех API-маршрутов и маршрутов веб-интерфейса, что исключает конфликты при совпадении имён заглушек с системными эндпоинтами.

Для каждой заглушки поддерживается отдельный постоянный `httpx.AsyncClient`[21] с пулом соединений (до 1000 одновременных, до 200 `keep-alive` с TTL 60 с). Класс `ProxycClientRegistry` управляет их созданием и закрытием; создание защищено `asyncio.Lock` против гонки при конкурентных запросах к новой заглушке; параметр `follow_redirects=False` гарантирует прозрачную передачу редиректов клиенту. Реализация приведена в листинге 2.9.

Листинг 2.9 – Реестр HTTP-клиентов ProxyClientRegistry

```
1 class ProxyClientRegistry:
2     """Один httpx.AsyncClient на заглушку с пулом соединений."""
3     def __init__(self) -> None:
4         self._clients: dict[str, httpx.AsyncClient] = {}
5         self._lock = asyncio.Lock()
6     async def get_or_create(
7         self,
8         mock_name: str,
9         base_url: str,
10        timeout: float | httpx.Timeout,
11    ) -> httpx.AsyncClient:
12        async with self._lock:
13            existing = self._clients.get(mock_name)
14            if existing is not None:
15                return existing
16            limits = httpx.Limits(
17                max_connections=1000,
18                max_keepalive_connections=200,
19                keepalive_expiry=60.0)
20            t = timeout if isinstance(timeout, httpx.Timeout) else
21                ↪ httpx.Timeout(timeout)
22            client = httpx.AsyncClient(
23                base_url=base_url.rstrip("/"),
24                timeout=t,
25                limits=limits,
26                follow_redirects=False)
27            self._clients[mock_name] = client
28            return client
29    async def remove(self, mock_name: str) -> None:
30        async with self._lock:
31            client = self._clients.pop(mock_name, None)
32            if client is not None:
33                await client.aclose()    # корректное закрытие всех
34                ↪ соединений пула
```

2.5.2 Конвейер обслуживания запроса

Прокси-роутер регистрирует два catch-all маршрута: `/ {mock_name}` и `/ {mock_name} / {path:path}` – оба делегируются единой функции `_handle_proxy`. Конвейер обслуживания: (1) HMGET пяти полей заглушки из Redis; (2) проверка статуса (RUNNING, иначе HTTP 503); (3) чтение глобальных параметров (`settings:global`); (4) проверка Rate Limiter (при превышении – HTTP 429); (5) инкремент счётчика успешных запросов; (6) получение клиента из `ProxyClientRegistry`; (7) чтение тела (кроме GET/HEAD/OPTIONS); (8) вызов `proxy_request` и возврат ответа; (9) сохранение метрик задержки в Redis.

Прозрачность достигается следующим образом: метод, тело и query string передаются upstream без изменений; заголовки пересылаются полностью за исключением Host и Content-Length, которые формируются заново для нового соединения. Ошибки транспортного уровня унифицированы: `httpx.TimeoutException` – HTTP 504, любой `httpx.RequestError` – HTTP 502. Все промежуточные этапы фиксируются в словаре `timeline` с точностью 0.001 мс, что позволяет на уровне Prometheus-метрик разделить задержку Redis, Rate Limiter и upstream.

2.5.3 Алгоритм Rate Limiting: Fixed Window Counter

Алгоритм реализован в классе `RateLimiter`: для каждой заглушки и каждого временного окна хранится счётчик по ключу `rate:{filename}:{window}`. Метка окна – целочисленное деление Unix-timestamp на размер окна. Проверка требует двух атомарных операций Redis: INCR и (только при создании нового счётчика) EXPIRE. Реализация приведена в листинге 2.10.

Листинг 2.10 – Реализация алгоритма Fixed Window Counter

```
1 class RateLimiter:
2     """Ограничение частоты запросов на заглушку в фиксированных
        ↪ временных окнах."""
3
4     def __init__(self, redis: Redis) -> None:
5         self._redis = redis
6
7     async def check(self, filename: str, limit: int,
8                     window_size: int = 1) -> bool:
9         """Инкрементирует счётчик текущего окна.
10            Возвращает True, если запрос в пределах лимита."""
11         window = int(time.time()) // window_size
12         key = f"rate:{filename}:{window}"
13
14         # Атомарный инкремент: Redis гарантирует неделимость
15            ↪ операции
16         count = await self._redis.incr(key)
17
18         # TTL устанавливается только при создании нового счётчика
19            ↪ (count == 1)
20         if count == 1:
21             await self._redis.expire(key, window_size * 2)
22
23         return count <= limit # True -> разрешить, False -> HTTP
24            ↪ 429
```

Атомарность INCR[25] гарантирует корректный подсчёт при конкурентных тысячах запросов без блокировок на стороне приложения. TTL равен удвоенному размеру окна и устанавливается только при первом запросе нового окна, что обеспечивает автоочистку устаревших ключей.

Алгоритм Fixed Window Counter допускает кратковременный всплеск до $2 \times limit$ на границе двух окон. Более точные альтернативы (Sliding Window Log, Sliding Window Counter) требуют хранения меток каждого запроса или двух счётчиков со взвешенным усреднением, что увеличивает на-

грузку на Redis пропорционально трафику. В контексте нагрузочного тестирования задача Rate Limiter – защита заглушки от перегрузки, а не точное соблюдение квоты с субсекундной точностью; статистические показатели теста рассчитываются на интервалах в несколько секунд. Таким образом, Fixed Window Counter является оптимальным выбором по соотношению точности и операционной стоимости.

2.5.4 Управление Rate Limiter через API

Включение и отключение Rate Limiter производится в реальном времени через PATCH-эндпоинт без остановки заглушки – операция обновляет единственное поле `rate_limit_enabled` в Hash-записи Redis; новое значение применяется с первого же следующего запроса. Порог `rate_limit` изменяется аналогично и вступает в силу с начала следующего временного окна. Эндпоинты управления приведены в таблице 11.

Таблица 11 – Эндпоинты управления Rate Limiter

Метод	Путь	Описание
PATCH	<code>/api/v1/mocks/{filename}/rate-limit</code>	Включить или отключить Rate Limiter (тело: <code>{"enabled": true/false}</code>)
PATCH	<code>/api/v1/mocks/{filename}/config</code>	Обновить порог лимита (<code>rate_limit</code>) и/или JVM-аргументы

2.6 Проектирование и программная реализация пользовательского интерфейса

2.6.1 Технологический подход и структура шаблонов

Веб-интерфейс реализован по технологии Server-Side Rendering с применением шаблонизатора Jinja2[19], встроенного в экосистему FastAPI. При каждом обращении к странице сервер формирует полный HTML-документ с актуальными данными из Redis и передаёт браузеру в готовом виде. Подход устраняет зависимость от внешних JavaScript-фреймворков и CDN-ресурсов и обеспечивает корректный рендер страницы даже при отключённом JavaScript. Динамическое обновление таблиц и журналов реализовано стандартным JavaScript без сторонних библиотек: страница периодически опрашивает REST API и точно обновляет DOM. Весь клиентский код сосредоточен в одном файле `web/static/js/app.js`, стили – в `web/static/css/style.css`; оба обслуживаются FastAPI через `StaticFiles`.

Все страницы наследуют базовый шаблон `base.html`: повторяющаяся разметка (боковая панель, заголовочная полоса, подключение стилей и сценариев) определяется единожды; дочерние шаблоны переопределяют только блоки переменного содержания через `{% extends %}` / `{% block %}`. Состав шаблонного пакета приведён в таблице 12.

Таблица 12 – Шаблоны веб-интерфейса

Файл	Маршрут	Назначение
<code>base.html</code>	—	Общая оболочка: боковая навигация, заголовочная полоса, индикатор Redis
<code>dashboard.html</code>	GET /	Таблица заглушек, сводные карточки, диалог регистрации

Файл	Маршрут	Назначение
settings.html	GET /settings	Управление хостами и учётными записями SSH
logs.html	GET /logs	Блок вывода журналов Java-процессов

Маршруты веб-интерфейса обслуживаются FastAPI-роутером `web.routes`, подключённым первым – до API-маршрутов и прокси-роутера. Каждый маршрут выполняет минимальную выборку данных из Redis через конвейер команд (`transaction=False`): команды `HGETALL` для всех заглушек отправляются одним пакетом и считываются единственным вызовом `pipe.execute()`, что исключает пропорциональный рост числа сетевых обращений. Реализация маршрута `Dashboard` приведена в листинге 2.11.

Листинг 2.11 – Маршрут `Dashboard`: конвейерная выборка заглушек из Redis

```

1 @router.get("/", include_in_schema=False)
2 async def dashboard_page(request: Request, redis: RedisClientDep,
   ↪ ...):
3     names = sorted(await redis.smembers("mocks:registry"))
4     if names:
5         # Pipeline: одна сетевая транзакция вместо N отдельных
   ↪ HGETALL
6         async with redis.pipeline(transaction=False) as pipe:
7             for name in names:
8                 pipe.hgetall(f"mocks:{name}")
9                 rows = await pipe.execute()
10    return templates.TemplateResponse(request=request,
   ↪ name="dashboard.html",
11        context={"mocks": mocks, "mocks_total": len(mocks),
12                "mocks_running": running, "redis_connected": ...,
13                "active_page": "dashboard"})

```


2.6.2 Базовый шаблон: навигация и индикатор Redis

Базовый шаблон реализует двухколоночный макет: фиксированная боковая панель шириной 220 px (`.sidebar`) и основная рабочая область (`.main-area`). Активный пункт навигации определяется переменной контекста `active_page`, передаваемой из каждого маршрута: шаблон добавляет CSS-класс `is-active` к соответствующей ссылке через условие Jinja2 – без клиентского кода.

В правом верхнем углу заголовочной полосы отображается индикатор состояния Redis: зелёная точка и «Redis: подключён» при успешном ответе на PING, красная и «Redis: отключён» при сбое. Состояние передаётся в переменную контекста `redis_connected` и рендерится через условный блок `{% if %}` с атрибутом `role="status"` для программ экранного доступа.

2.6.3 Главная панель управления

Верхняя часть Dashboard содержит четыре сводные карточки: всего заглушек, запущено (RUNNING), аварийных (ERROR), хостов доступно – для оценки состояния инфраструктуры без анализа таблицы. Таблица заглушек включает шесть столбцов: имя файла (JetBrains Mono), целевой хост, цветной статусный бейдж, TCP-порт, тумблер Rate Limiter и кнопки управления. Бейджи реализованы через CSS-классы `badge-running`, `badge-error`, `badge-registered`, `badge-stopped`: цвет однозначно кодирует состояние без чтения текста. Кнопки «Старт» и «Стоп» взаимоисключающи – шаблон отображает одну из них по текущему статусу.

Тумблер Rate Limiter реализован как `<input type="checkbox">` со специализированной CSS-разметкой `.toggle`; изменение состояния немедленно отправляет PATCH-запрос к `/api/v1/mocks/{filename}/rate-limit` без перезагрузки страницы. Таблица обновляется каждые 5 секунд через `setInterval` (интервал задаётся атрибутом `data-poll-interval-ms="5000"`): параллельные запросы к `/api/v1/mocks` и `/api/v1/hosts`

выполняются через `Promise.all`, полученные данные точно обновляют DOM без полного перестроения таблицы. Реализованная главная панель представлена на рисунке 12.

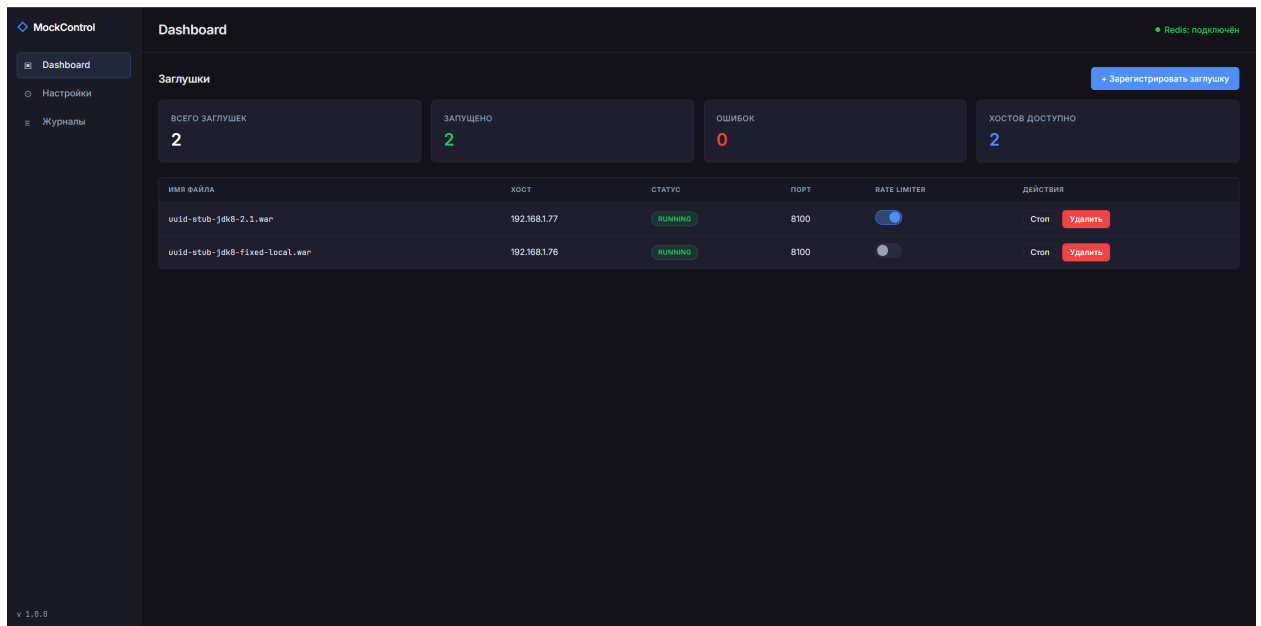


Рисунок 12 – Реализованная главная панель управления

2.6.4 Диалоговое окно регистрации заглушки

Регистрация выполняется через диалоговое окно поверх Dashboard без перехода на отдельную страницу, сохраняя контекст. Форма содержит пять полей, приведённых в таблице 13.

Таблица 13 – Поля формы регистрации заглушки

Поле	Тип	Описание
file	Область перетаскивания	.jar/.war до 512 МБ; поддерживает drag-and-drop
hostname	Раскрывающийся список	Целевой хост из реестра; список формируется сервером при рендере
jvm_args	Многострочное поле	JVM-аргументы; моноширинный шрифт

Поле	Тип	Описание
rate_limit	Числовое поле	Лимит запросов в секунду; 0 – без ограничений
auto_start	Флажок	Запустить заглушку немедленно после регистрации

Форма отправляется как `multipart/form-data` на `POST /api/v1/mocks`. Сервер отвечает кодом 202 (Accepted): SFTP-передача файла выполняется в фоне через `BackgroundTasks` FastAPI, что исключает зависание браузера при передаче крупных артефактов. После ответа 202 клиентский код закрывает диалог и через 1 секунду инициирует принудительное обновление таблицы. Область перетаскивания реализована на стандартных событиях `dragover/dragleave/drop`: при наведении рамка подсвечивается акцентным цветом, при сбросе отображается имя файла. Реализованное диалоговое окно представлено на рисунке 13.

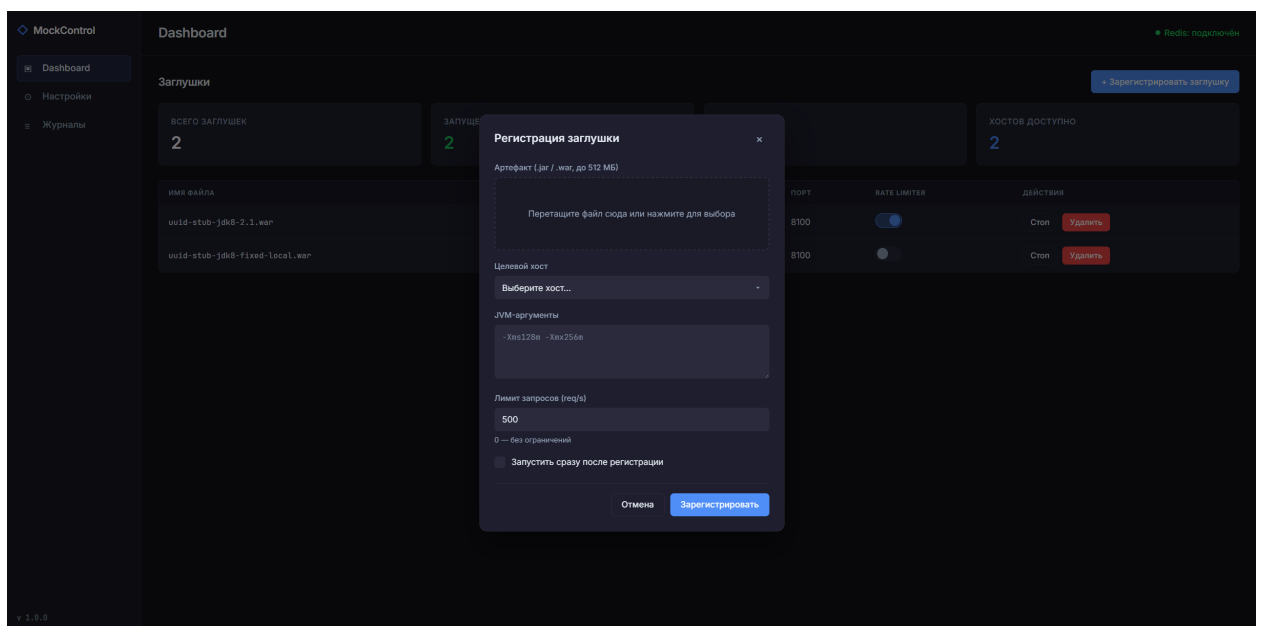


Рисунок 13 – Реализованное диалоговое окно регистрации заглушки

2.6.5 Страница настроек: хосты и учётные записи

Страница организована в две вкладки («Хосты» и «Учётные записи»), переключаемые JavaScript без перезагрузки и реализованные по специфика-

ции ARIA (role="tablist", role="tab", role="tabpanel"): при выборе вкладки устанавливаются aria-selected и CSS-класс is-active.

Вкладка «Хосты» отображает таблицу целевых серверов: имя хоста, порт SSH, рабочую директорию, статус SSH-соединения (AVAILABLE/UNAVAILABLE/UNKNOWN), дату последней проверки и описание. Все параметры хоста хранятся в атрибутах data-* строки таблицы: при открытии формы редактирования клиентский код считывает их и заполняет поля без дополнительного запроса к API.

Вкладка «Учётные записи» отображает список учётных записей Linux. Пароли скрыты строкой (aria-hidden="true" для программ экранного доступа) – это предотвращает случайную утечку при демонстрации системы. Пароль шифруется на сервере алгоритмом AES-128 (Fernet) перед записью в Redis. Реализованные страницы настроек представлены на рисунках 14 и 15.

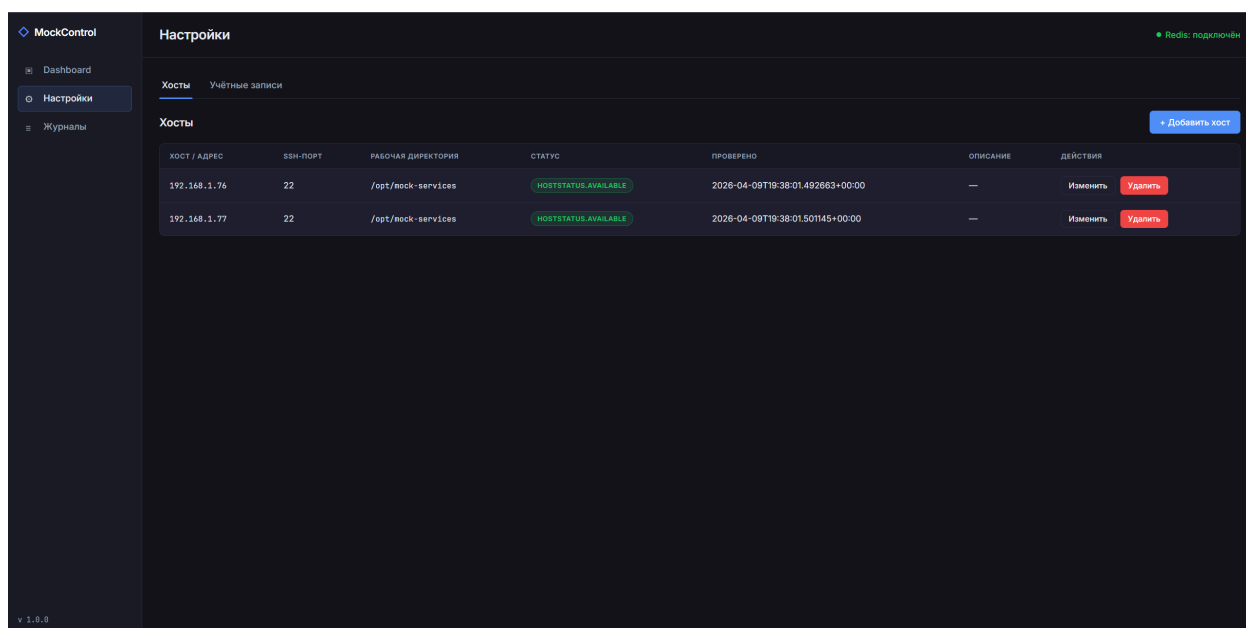


Рисунок 14 – Реализованная страница настроек, вкладка «Хосты»

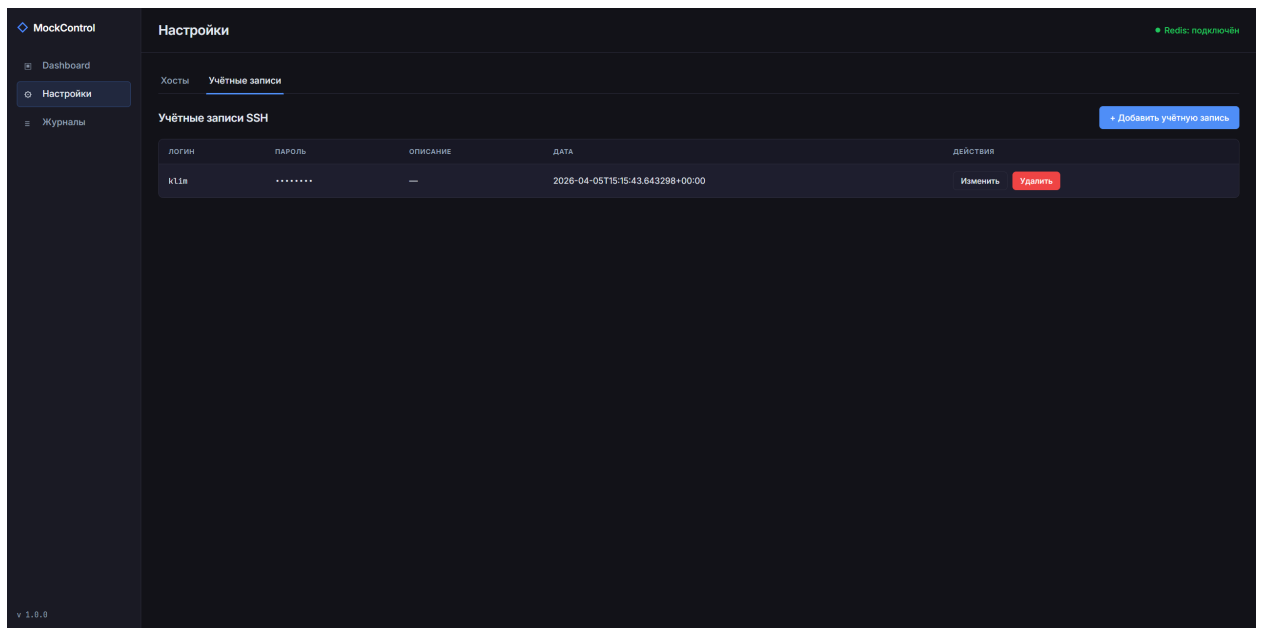


Рисунок 15 – Реализованная страница настроек, вкладка «Учётные записи»

2.6.6 Страница журналов процессов

Страница предназначена для просмотра stdout/stderr Java-процессов в режиме, близком к реальному времени. Выбор заглушки – через раскрывающийся список; при смене выбора клиент последовательно запрашивает `GET /api/v1/mocks/{filename}` (текущие PID и порт) и `GET /api/v1/logs/{filename}` (строки журнала). Строка состояния над блоком вывода отображает имя файла, PID и порт процесса.

Блок вывода (`.terminal`) реализован как `<div>` с тёмным фоном `#0D0D0D` и шрифтом JetBrains Mono. Функция `formatLogLine` выделяет временную метку регулярным выражением (ISO 8601 или в квадратных скобках) и окрашивает её в `#64748B`; строки с `WARN` получают жёлтый фон и класс `terminal__line-warn`, с `ERROR` – красный фон и класс `terminal__line-error`. Автообновление реализовано через `setInterval` с интервалом 2 секунды, управляемым флажком «Авто». Кнопка «Очистить» вызывает `DELETE /api/v1/logs/{filename}`, что сбрасывает буфер в Redis-ключе `logs:{filename}`. Реализованная страница представлена на рисунке 16.

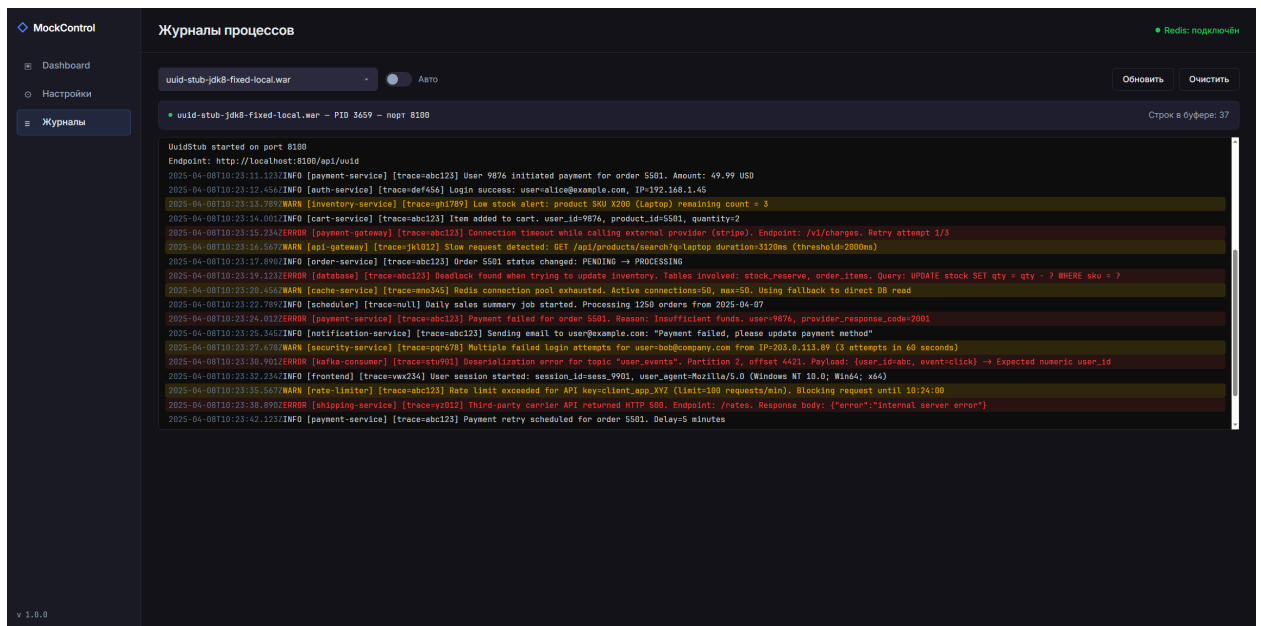


Рисунок 16 – Реализованная страница просмотра журналов процессов

2.6.7 Делегирование событий, взаимодействие с API и система стилей

Клиентский код не использует библиотек привязки событий. Обработчики `click` и `change` устанавливаются на корневые элементы страниц; тип действия определяется атрибутом `data-action` ближайшего подходящего элемента. Динамически добавляемые строки таблицы при периодическом обновлении немедленно обрабатываются существующими обработчиками без дополнительной регистрации.

Все обращения к REST API выполняются через `fetch()` с унифицированной обработкой ошибок: поле `detail` из JSON-ответа отображается в диалоге предупреждения. Кнопки на время запроса переводятся в неактивное состояние через `setButtonLoading`: разметка сохраняется в `dataset.origHtml` и заменяется индикатором загрузки, атрибут `disabled` блокирует повторное нажатие.

Все цветовые и типографические константы определены как CSS-переменные в секции `:root` файла `style.css`. Оба шрифта – Inter и JetBrains Mono – подключаются через `@font-face` из локальной директории `web/static/fonts/` без внешних CDN, что соответствует требованию

технологической независимости (раздел 1.3). Ключевые переменные оформления приведены в таблице 14.

Таблица 14 – Переменные системы визуального оформления интерфейса

Переменная	Значение	Назначение
<code>--bg-main</code>	<code>#121218</code>	Основной фон рабочей области
<code>--bg-sidebar</code>	<code>#1A1A24</code>	Фон боковой панели навигации
<code>--bg-card</code>	<code>#1E1E2E</code>	Фон карточек и диалоговых окон
<code>--accent</code>	<code>#4F8EF7</code>	Акцентный цвет кнопок и активных элементов
<code>--text-primary</code>	<code>#E2E8F0</code>	Основной текст
<code>--text-secondary</code>	<code>#94A3B8</code>	Метки, подсказки
<code>--status-running</code>	<code>#22C55E</code>	Статус RUNNING
<code>--status-registered</code>	<code>#EAB308</code>	Статусы REGISTERED / STOPPED
<code>--status-error</code>	<code>#EF4444</code>	Статус ERROR / UNAVAILABLE
<code>--bg-terminal</code>	<code>#0D0D0D</code>	Фон блока вывода журналов

2.7 Реализация механизма экспорта метрик мониторинга

2.7.1 Общая схема и состав метрик

Механизм мониторинга построен по pull-модели Prometheus[23]: внешняя система периодически опрашивает эндпоинт `GET /metrics`, а управляющее приложение возвращает актуальный снимок показателей в формате Prometheus exposition format[24]. Метрические данные хранятся в Redis и обновляются в процессе обслуживания каждого проксированного запроса: функция `collect_metrics` при обращении к `/metrics` не вычисляет значения, а лишь считывает уже агрегированные данные за одну пакетную операцию.

Реализация разделена на две части: функция накопления `record_proxy_observation`, вызываемая в конце каждого проксиро-

ванного запроса, и функция экспорта `collect_metrics`, вызываемая по каждому запросу к `/metrics`. Система экспортирует двенадцать метрических серий двух типов по спецификации Prometheus: счётчики (counter, монотонно возрастают) и измерители (gauge, текущее мгновенное значение). Полный состав метрик приведён в таблице 15.

Таблица 15 – Экспортируемые метрики системы

Имя метрики	Тип	Описание
<code>mockcontrol_mocks_total</code>	gauge	Число зарегистрированных заглушек
<code>mockcontrol_mocks_running</code>	gauge	Число заглушек в состоянии RUNNING
<code>mockcontrol_mocks_errors</code>	gauge	Число заглушек в состоянии ERROR
<code>mockcontrol_hosts_available</code>	gauge	Число хостов в состоянии AVAILABLE
<code>mockcontrol_proxy_requests_total</code>	counter	Успешно проксированных запросов (метка <code>mock</code>)

Имя метрики	Тип	Описание
mockcontrol_rate_limit_rejected_total	counter	Отклонённых запросов по Rate Limiter (метка mock)
mockcontrol_proxy_stage_latency_ms_sum	counter	Накопленная сумма задержек по этапам, мс (метки mock, stage)
mockcontrol_proxy_stage_latency_ms_count	counter	Число наблюдений задержки по этапам (метки mock, stage)
mockcontrol_proxy_stage_latency_ms_avg	gauge	Среднее время этапа, мс (метки mock, stage)

Имя метрики	Тип	Описание
mockcontrol_proxy_stage_latency_ms_last	gauge	Последнее измеренное время этапа, мс (метки mock, stage)
mockcontrol_proxy_outcome_total	counter	Исходы проксирования по типу (метки mock, outcome)
mockcontrol_proxy_upstream_status_total	counter	Ответы заглушки по HTTP-коду (метки mock, status)

Все метрики с меткой `mock` дополнительно агрегируются по значению `mock="__all__"`, что позволяет получать суммарные показатели по всем заглушкам единым Prometheus-запросом. Метка `stage` принимает значения этапов функции `_handle_proxy`: `redis_mock_hmget_ms`, `rate_limiter_check_ms`, `httpx_proxy_request_ms`, `proxy_total_ms` и др. Метка `outcome` принимает семь значений: `ok`, `timeout`, `request_error`, `not_found`, `not_running`, `invalid_target`, `rate_limited`.

2.7.2 Накопление метрик и двухпроходная сборка

По завершении каждого запроса функция `record_proxy_observation` обновляет показатели Redis через единственный конвейер (`transaction=False`): для каждого этапа выполняются `HINCRBYFLOAT` (сумма задержек), `HINCRBY` (счётчик наблюдений) и `HSET` (последнее значение) – как для ключа конкретной заглушки, так и для агрегирующего ключа `__all__`. Параметр `transaction=False` использован намеренно: частичная видимость промежуточных значений допустима, поскольку Prometheus считывает метрики с интервалом в десятки секунд.

Функция `collect_metrics` собирает все данные за два конвейерных прохода. В первом запрашиваются реестры `mocks:registry` и `hosts:registry`; во втором – единым пакетом: `HGETALL` каждой заглушки, `HGET` статуса каждого хоста и семь метрических ключей на заглушку плюс пять агрегирующих ключей `__all__`. Число сетевых транзакций фиксировано: два round-trip вне зависимости от числа заглушек. Реализация приведена в листинге 2.12.

Листинг 2.12 – Двухпроходная конвейерная сборка метрик из Redis

```
1 async def collect_metrics(redis: Redis) -> str:
2     # Проход 1: реестры
3     pipe = redis.pipeline(transaction=False)
4     pipe.smembers("mocks:registry")
5     pipe.smembers("hosts:registry")
6     reg_results = await pipe.execute()
7     mock_files = sorted(reg_results[0] or [])
8     host_names = sorted(reg_results[1] or [])

9     # Проход 2: данные заглушек, хостов и все метрические ключи
10    pipe2 = redis.pipeline(transaction=False)
11    for filename in mock_files:
12        pipe2.hgetall(f"mocks:{filename}")           # статус заглушки
13    for hostname in host_names:
14        pipe2.hget(f"hosts:{hostname}", "status")    # статус хоста
15    for filename in mock_files:
16        pipe2.get(f"metrics:proxy_total:{filename}")
17        pipe2.get(f"metrics:rejected_total:{filename}")
18        pipe2.hgetall(f"metrics:proxy_stage_sum_ms:{filename}")
19        pipe2.hgetall(f"metrics:proxy_stage_count:{filename}")
20        pipe2.hgetall(f"metrics:proxy_stage_last_ms:{filename}")
21        pipe2.hgetall(f"metrics:proxy_outcome_total:{filename}")
22        pipe2.hgetall(f"metrics:proxy_upstream_status_total:{filename}
23        ↪      ame}")
24    # Агрегаты __all__
25    pipe2.hgetall("metrics:proxy_stage_sum_ms:__all__")
26    pipe2.hgetall("metrics:proxy_stage_count:__all__")
27    pipe2.hgetall("metrics:proxy_stage_last_ms:__all__")
28    pipe2.hgetall("metrics:proxy_outcome_total:__all__")
29    pipe2.hgetall("metrics:proxy_upstream_status_total:__all__")
30    batch = await pipe2.execute()
31    # ... формирование строк текстового ответа ...
```

После получения пакета функция разбирает результаты по позиционному индексу: первые `n_m` элементов – Hash-записи заглушек, следующие

n_h – статусы хостов, оставшиеся – метрические ключи в порядке добавления в конвейер. Ответ формируется по спецификации Prometheus exposition format 0.0.4[24] (text/plain; version=0.0.4; charset=utf-8): каждая серия открывается строками # HELP и # TYPE, после чего следуют строки значений. Значения меток экранируются функцией _escape_label_value (обратный слеш, перевод строки, двойные кавычки), что обеспечивает корректный разбор при наличии специальных символов в именах файлов заглушек. Фрагмент ответа приведён в листинге 2.13.

Листинг 2.13 – Фрагмент ответа эндпоинта /metrics

```
1 mockcontrol_proxy_stage_latency_ms_count{mock="__all__",stage="respon_
  ↳onse_build_ms"} 158287
2 mockcontrol_proxy_stage_latency_ms_avg{mock="__all__",stage="respo_
  ↳nse_build_ms"} 0.008
3 mockcontrol_proxy_stage_latency_ms_last{mock="__all__",stage="resp_
  ↳onse_build_ms"} 0.008
4 mockcontrol_proxy_outcome_total{mock="uuid-stub-jdk8-2.1.war",outc_
  ↳ome="ok"} 1
5 mockcontrol_proxy_outcome_total{mock="uuid-stub-jdk8-fixed-local.w_
  ↳ar",outcome="ok"} 158270
6 mockcontrol_proxy_outcome_total{mock="__all__",outcome="not_found"}
  ↳ 3
```

2.8 Выводы по второй главе

Во второй главе выпускной квалификационной работы выполнены проектирование архитектуры, разработка структур данных и программная реализация всех ключевых компонентов разработанного программного комплекса.

На основании сравнительного анализа архитектурных стилей обоснован выбор монолитной архитектуры с явно выраженной внутренней модульной организацией. Показано, что данное решение оптимально соответствует требованиям проекта: исключает накладные расходы на межсервисное сете-

вое взаимодействие, устраняет зависимость от внешних систем оркестрации контейнеров и обеспечивает простоту развёртывания в закрытом корпоративном контуре. Разработана гибкая топология развёртывания, в рамках которой единый управляющий процесс координирует как локальные, так и удалённые целевые хосты по единому алгоритму через механизмы `asyncssh`/`SFTP` без установки дополнительного программного обеспечения на целевые серверы.

Архитектура системы формализована в нотации C4 Model на трёх уровнях детализации: контекстном, контейнерном и компонентном. Выделены и описаны шесть программных компонентов – Lifecycle Manager, Proxy Engine, Rate Limiter, Settings Module, Crypto Service и Metrics Exporter.

Спроектированы структуры данных Redis для хранения конфигураций заглушек, хостов и учётных записей в виде Hash-записей с обеспечением доступа за одну операцию `HGETALL`. Реализован алгоритм ограничения интенсивности запросов Fixed Window Counter на основе атомарной операции `INCR` с автоматической очисткой устаревших счётчиков через механизм TTL. Разработана стратегия восстановления состояния после перезапуска системы: проверка фактической живости каждого процесса с последующим обновлением Redis-записей до согласованного состояния.

Реализованы асинхронные механизмы управления жизненным циклом Java-процессов с применением `asyncio.create_subprocess_exec` и `asyncio.to_thread` для исключения блокирования событийного цикла на операциях ввода-вывода. Разработан механизм безопасного разбора пользовательских JVM-аргументов на основе лексического анализатора `shlex`, обеспечивающий защиту от инъекции команд оболочки при запуске процессов на удалённых хостах. Реализован пул постоянных SSH-соединений, снижающий задержку операций управления на удалённых хостах за счёт исключения повторного согласования ключей при каждой операции.

Разработан модуль проксирования запросов на основе асинхронного HTTP-клиента `httpx` с индивидуальным пулом соединений для каждой за-

глушки. Реализована прозрачная маршрутизация входящего трафика с сохранением метода, заголовков и тела запроса. Обоснована применимость алгоритма Fixed Window Counter для задач ограничения интенсивности запросов в среде нагрузочного тестирования с учётом его операционной стоимости и допустимой погрешности на границах временных окон.

Спроектирован и реализован веб-интерфейс на основе технологии формирования страниц на стороне сервера с применением шаблонизатора Jinja2. Интерфейс не требует внешних зависимостей: оба шрифта и весь клиентский код подключаются из локальных файлов без обращения к внешним источникам. Реализован механизм делегирования событий, обеспечивающий корректную обработку динамически добавляемых элементов таблицы без дополнительной регистрации обработчиков.

Реализован механизм экспорта метрик мониторинга в формате Prometheus exposition format. Разработана двухпроходная схема конвейерной сборки данных из Redis, обеспечивающая формирование ответа за фиксированное число сетевых транзакций вне зависимости от количества зарегистрированных заглушек. Определены двенадцать метрических серий с детализацией задержек по этапам конвейера обработки запроса, что обеспечивает возможность инструментальной диагностики узких мест инфраструктуры в ходе нагрузочного тестирования.

Полученные результаты образуют завершённую программную реализацию системы, готовую к развёртыванию в операционной среде и верификации в рамках тестирования, описанного в третьей главе.

3 ВНЕДРЕНИЕ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО КОМПЛЕКСА

3.1 Описание процесса развертывания системы в среде ОС Astra Linux (настройка Uvicorn/Gunicorn и службы Redis).

3.2 Тестирование механизмов автоматического восстановления состояний (персистентность Redis AOF) и перезапуска сервисов.

3.3 Экспериментальная оценка производительности прокси-модуля и эффективности алгоритма Rate Limiting под нагрузкой.

3.4 Анализ работы системы в кластерном режиме и оценка алгоритмов распределения нагрузки между узлами.

3.5 Рекомендации по дальнейшему развитию системы (шаблонизация конфигураций, Self-healing).

3.6 Выводы по третьей главе.

ЗАКЛЮЧЕНИЕ

Описание проделанной работы

В результате работы были выполнены следующие задачи:

1. Задача 1
2. Задача 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. WildFly — JBoss Application Server Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.wildfly.org/documentation/>.
2. Eclipse GlassFish Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://glassfish.org/documentation>.
3. asyncio — Asynchronous I/O. Python 3.12 Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://docs.python.org/3/library/asyncio.html>.
4. FastAPI Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://fastapi.tiangolo.com/>.
5. ASGI (Asynchronous Server Gateway Interface) Specification. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://asgi.readthedocs.io/en/latest/>.
6. Pydantic Documentation — Data validation using Python type hints. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://docs.pydantic.dev/>.
7. Uvicorn — An ASGI web server, for Python. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.uvicorn.org/>.
8. Redis Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://redis.io/docs/>.
9. Redis Persistence: Append Only File. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://redis.io/docs/management/persistence/>.
10. Astra Linux Special Edition — официальная документация. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://wiki.astralinux.ru/>.
11. WireMock Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://wiremock.org/docs/>.
12. MockServer Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://www.mock-server.com/>.

13. Mountebank Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <http://www.mbtest.org/docs/>.
14. Apache Tomcat 9 Documentation. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://tomcat.apache.org/tomcat-9.0-doc/index.html>.
15. Tomcat Clustering/Session Replication How-To. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. <https://tomcat.apache.org/tomcat-9.0-doc/cluster-howto.html>.
16. Apache Tomcat Configuration Reference: The Rate Limit Filter. – 2025. – [Электронный ресурс] Дата обращения: 09.01.2026. https://tomcat.apache.org/tomcat-9.0-doc/config/filter.html#Rate_Limit_Filter.
17. OpenJDK 17 Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://openjdk.org/projects/jdk/17/>.
18. The C4 model for visualising software architecture. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://c4model.com/>.
19. Jinja2 Documentation — The Python Template Engine. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://jinja.palletsprojects.com/>.
20. Paramiko Documentation — SSH2 protocol library for Python. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.paramiko.org/>.
21. HTTPX — A next-generation HTTP client for Python. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.python-httpx.org/>.
22. Cryptography Library — Fernet (symmetric encryption). – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://cryptography.io/en/latest/fernet/>.
23. Prometheus Documentation. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://prometheus.io/docs/introduction/overview/>.

24. Prometheus Exposition Formats. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. https://prometheus.io/docs/instrumenting/exposition_formats/.

25. Redis Commands: INCR — Increment the integer value of a key. – 2025. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://redis.io/commands/incr/>.

26. Web Content Accessibility Guidelines (WCAG) 2.1. – 2018. – [Электронный ресурс] Дата обращения: 15.02.2026. <https://www.w3.org/TR/WCAG21/>.

ПРИЛОЖЕНИЕ А